

CS 342: Computer Vision and Pattern Recognition

Instructor: Br. Tamal Mj

Shanti Mantra

Lyrics in Sanskrit

Meaning in English

ॐ सह नावतु।

Om, May we all be protected

सह नौ भुनक्तु।

May we all be nourished

सह वीर्यं करवावहै।

May we work together with great energy

तेजस्यि नावधीतमस्तु

May our intellect be sharpened (may our study be effective)

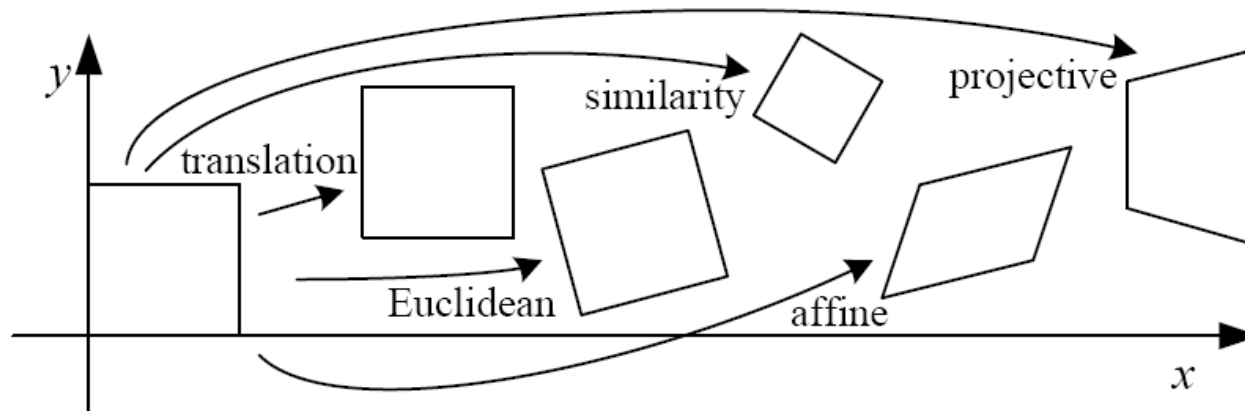
मा विद्विषावहै।

Let there be no Animosity amongst us

ॐ शान्तिः शान्तिः शान्तिः ॥

Om, peace (in me), peace (in nature), peace (in divine forces)

2D image transformations (reference table)



Name	Matrix	# D.O.F.	Preserves:	Icon
translation	$\begin{bmatrix} I & t \end{bmatrix}_{2 \times 3}$	2	orientation + ...	
rigid (Euclidean)	$\begin{bmatrix} R & t \end{bmatrix}_{2 \times 3}$	3	lengths + ...	
similarity	$\begin{bmatrix} sR & t \end{bmatrix}_{2 \times 3}$	4	angles + ...	
affine	$\begin{bmatrix} A \end{bmatrix}_{2 \times 3}$	6	parallelism + ...	
projective	$\begin{bmatrix} \tilde{H} \end{bmatrix}_{3 \times 3}$	8	straight lines	

‘Homography’

Projective Transformations

Projective transformations are combos of

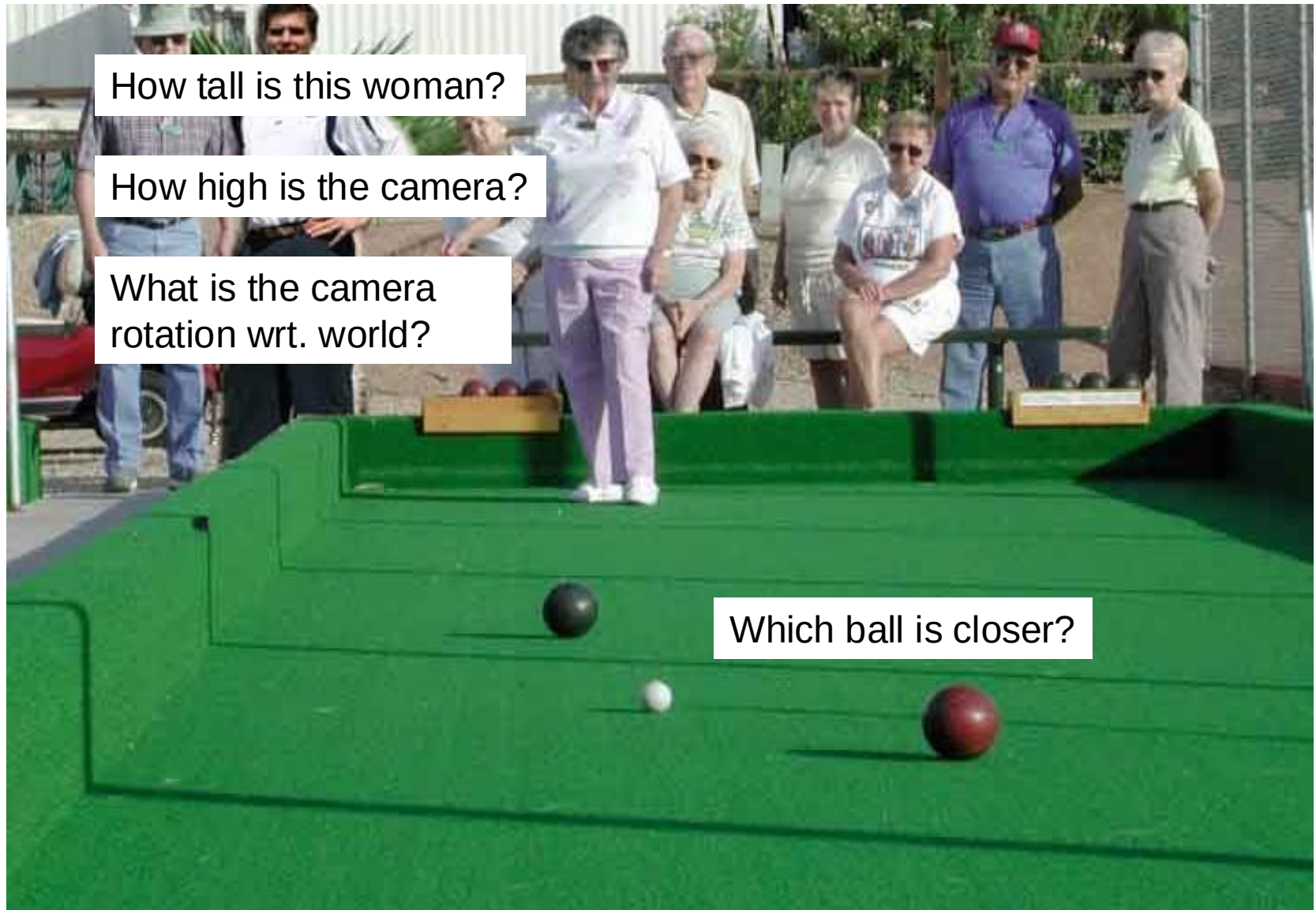
- Affine transformations, and
- Projective warps

$$\begin{bmatrix} x' \\ y' \\ w' \end{bmatrix} = \begin{bmatrix} a & b & c \\ d & e & f \\ g & h & i \end{bmatrix} \begin{bmatrix} x \\ y \\ w \end{bmatrix}$$

Properties of projective transformations:

- Lines map to lines
- Parallel lines do not necessarily remain parallel
- Ratios are not preserved
- Closed under composition
- Models change of basis
- Projective matrix is defined up to a scale (8 DOF)

Cameras and World Geometry



How tall is this woman?

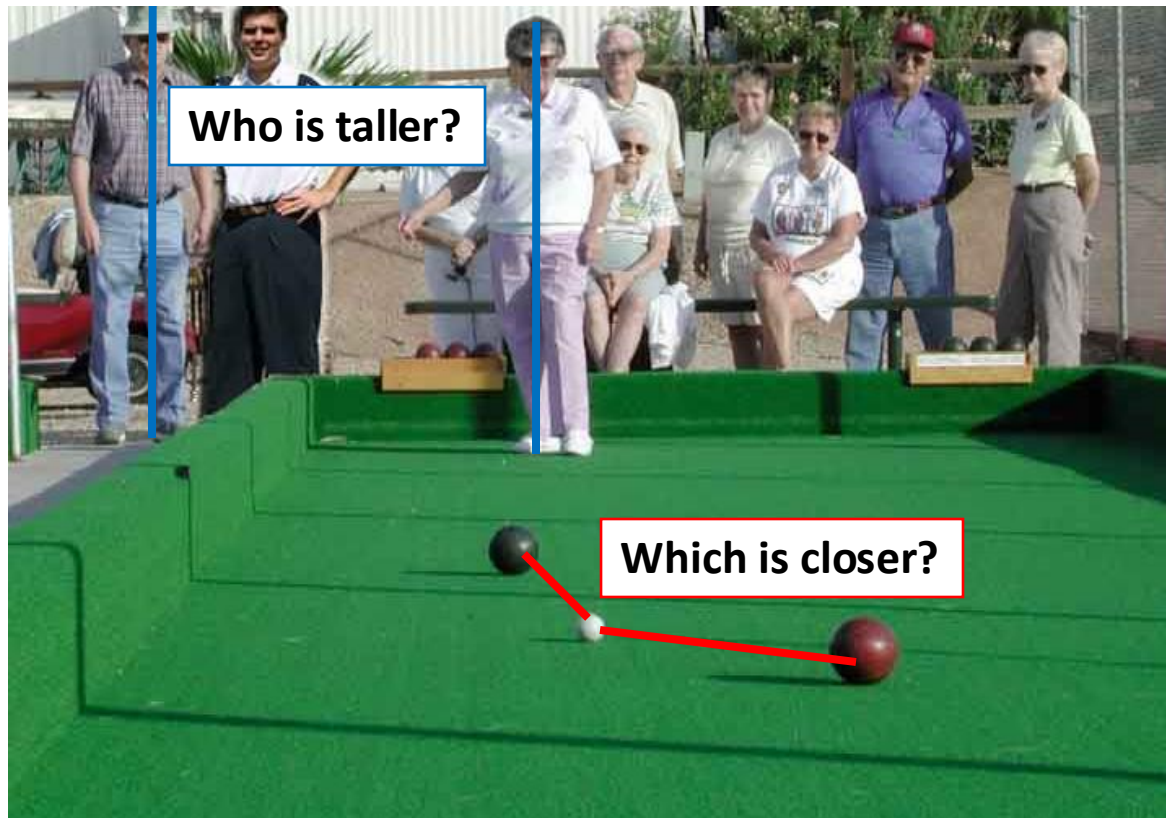
How high is the camera?

What is the camera
rotation wrt. world?

Which ball is closer?

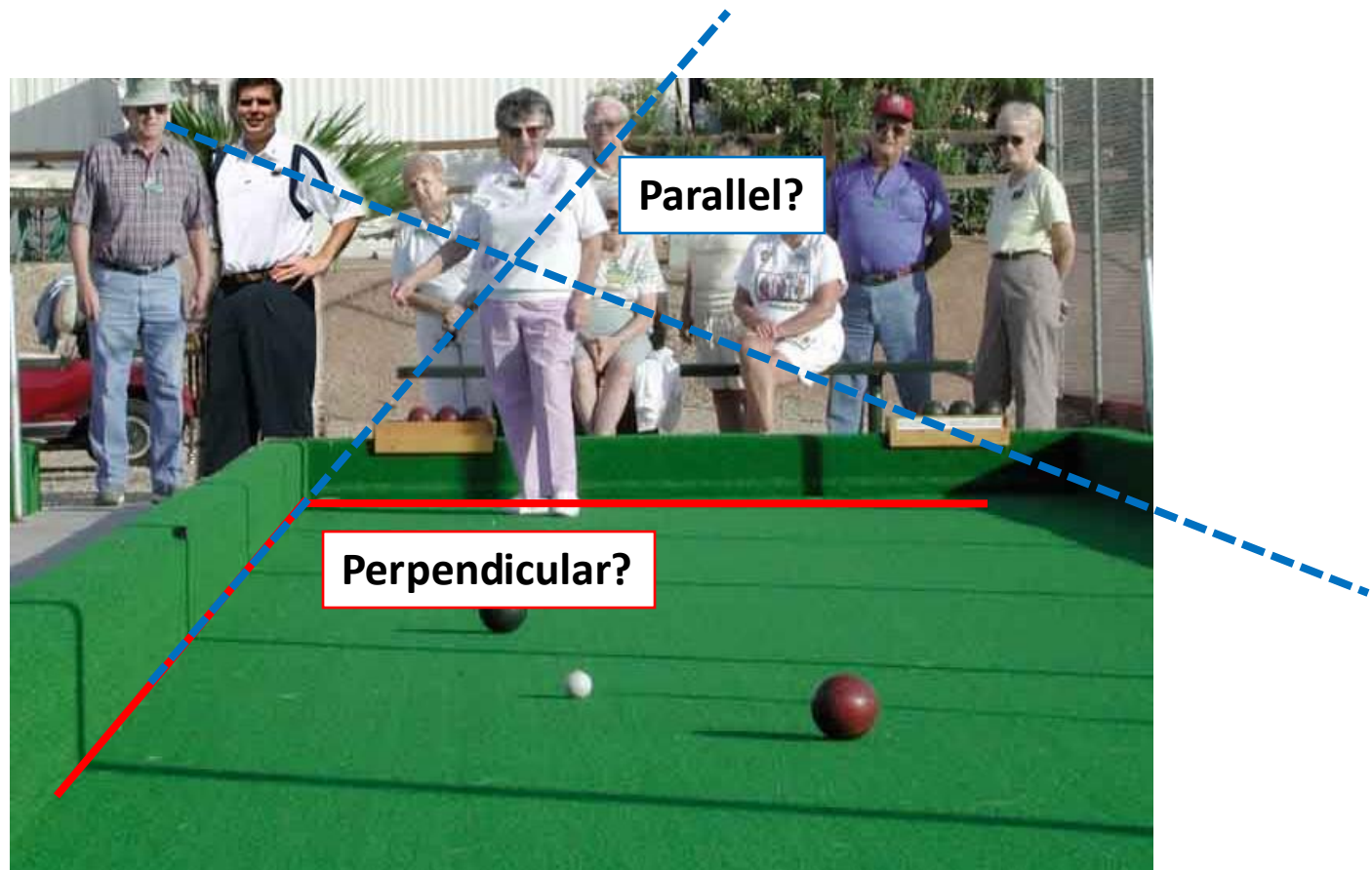
Projective Geometry

Length (and so area) is lost.



Projective Geometry

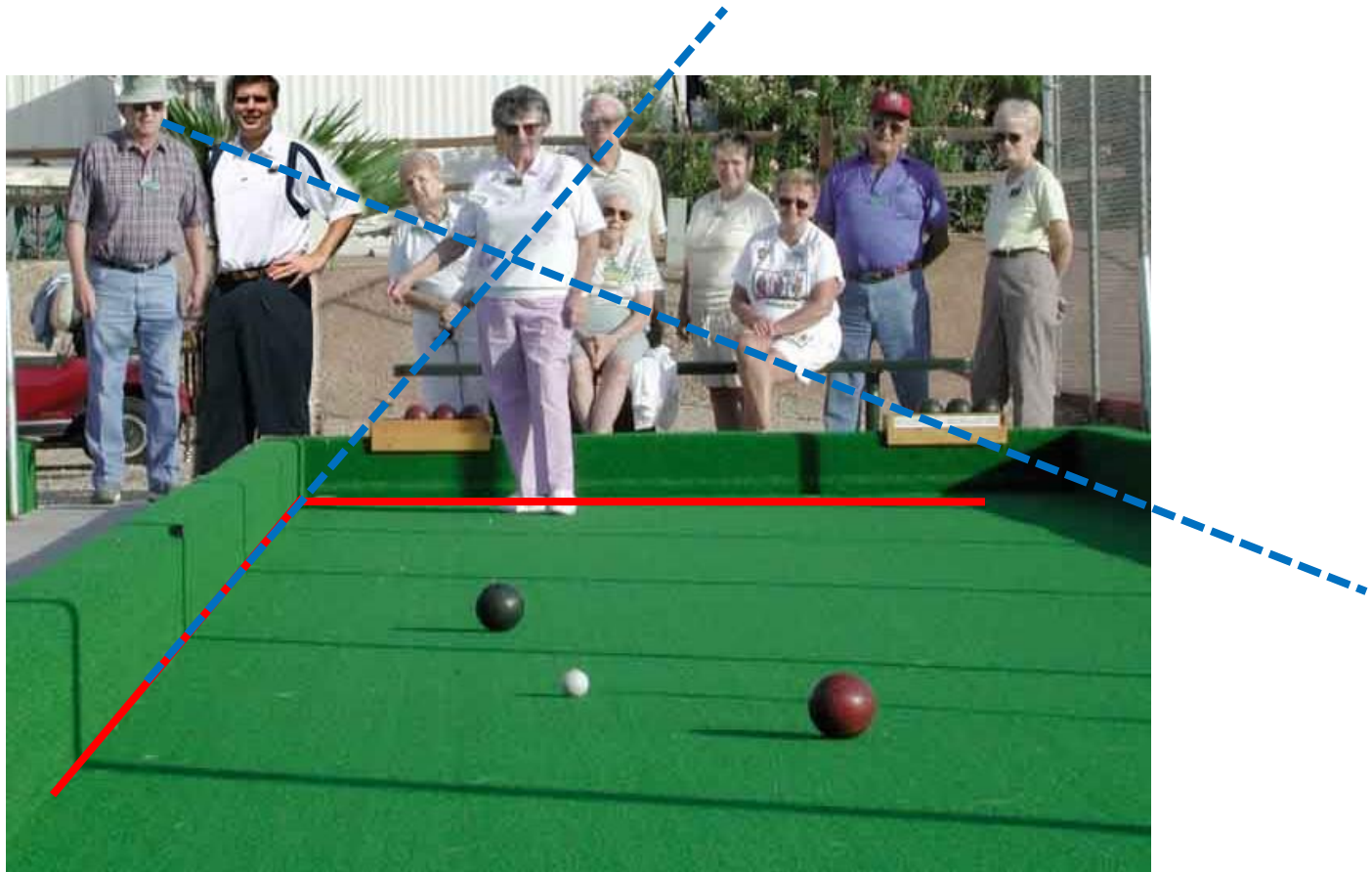
Angles are lost.



Projective Geometry

What is preserved?

- Straight lines are still straight.

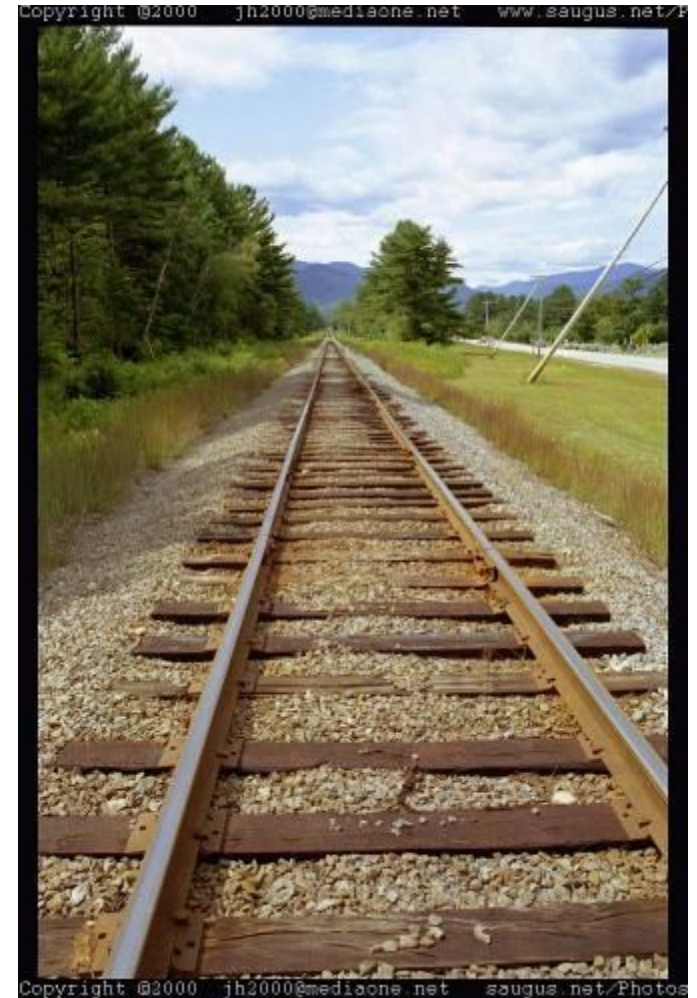
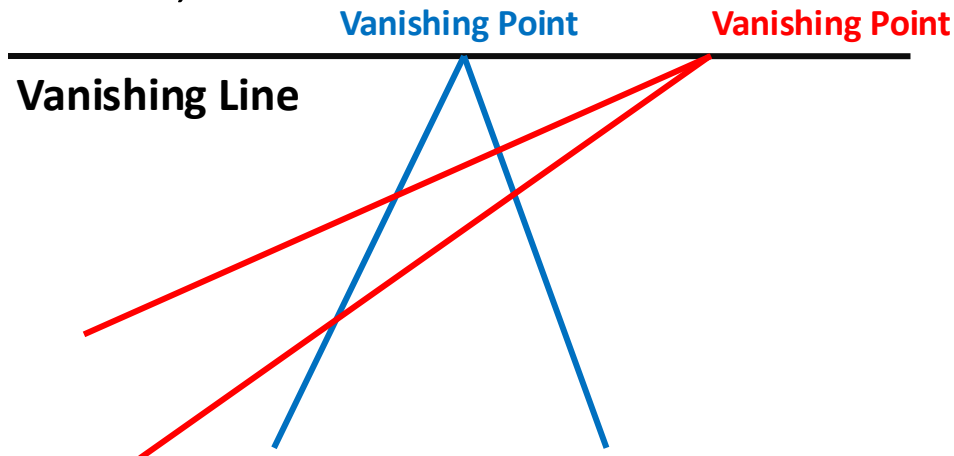


Vanishing points and lines

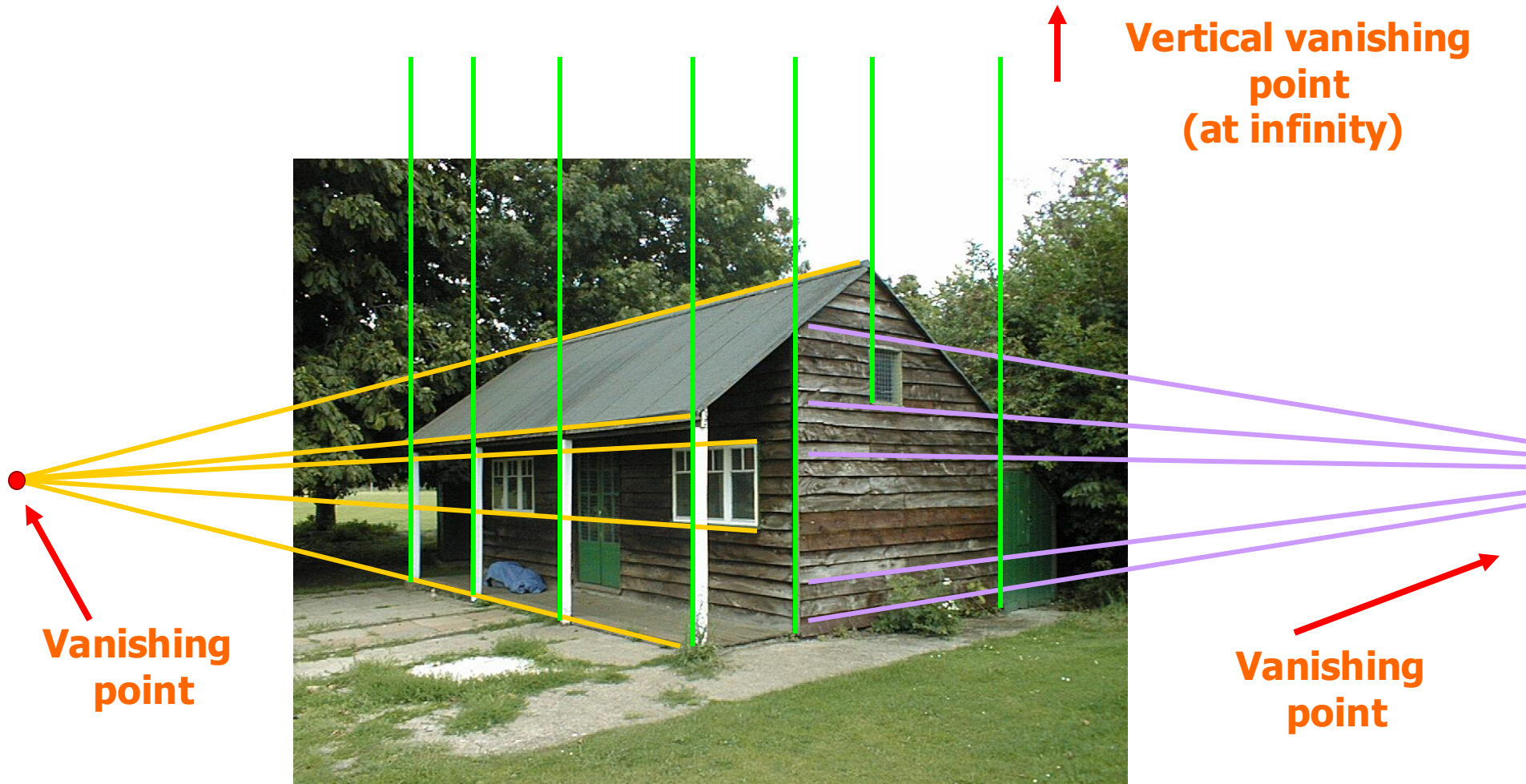
Parallel lines in the world intersect in the projected image at a “vanishing point”.

Parallel lines on the same plane in the world converge to vanishing points on a “vanishing line”.

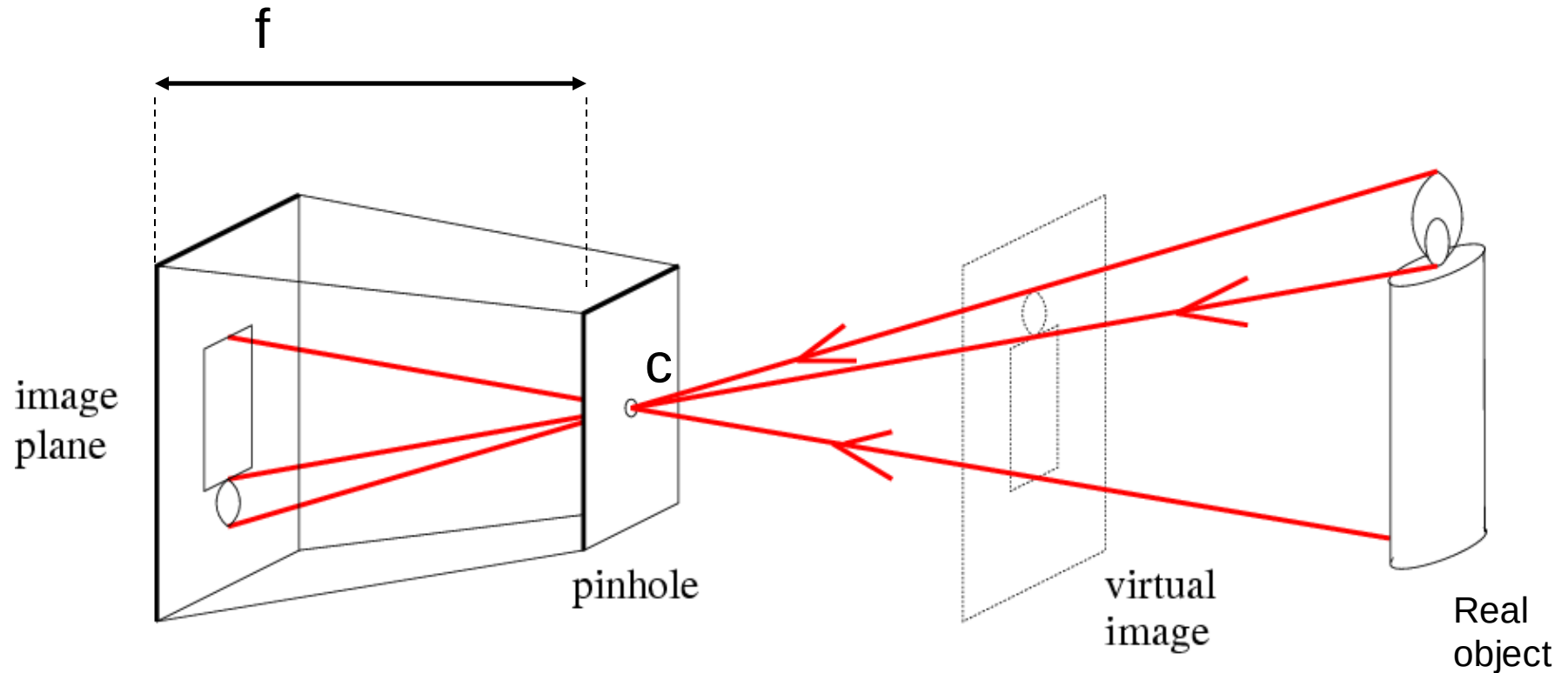
E.G., the horizon.



Vanishing points and lines



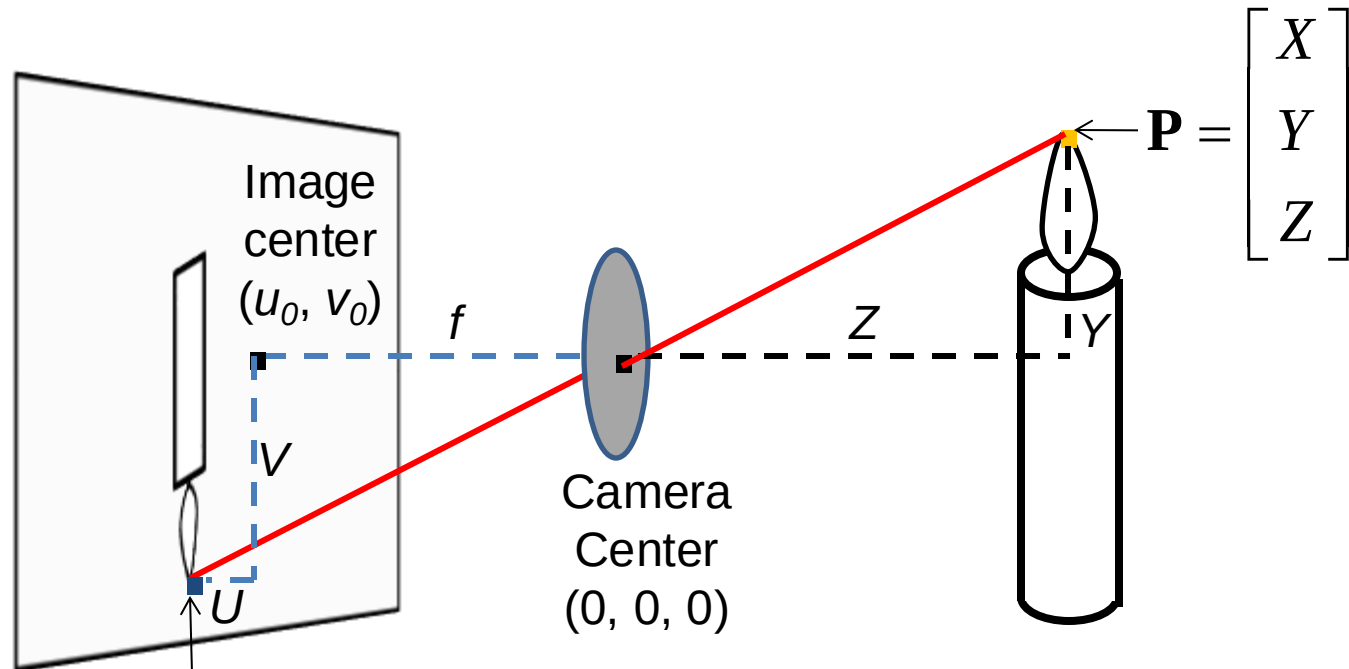
Pinhole camera model



f = Focal length

c = Optical center of the camera

Projection: world coordinates $\rightarrow$ image coordinates



$$\mathbf{p} = \begin{bmatrix} U \\ V \end{bmatrix}$$

$\mathbf{p}$ = distance from
image center

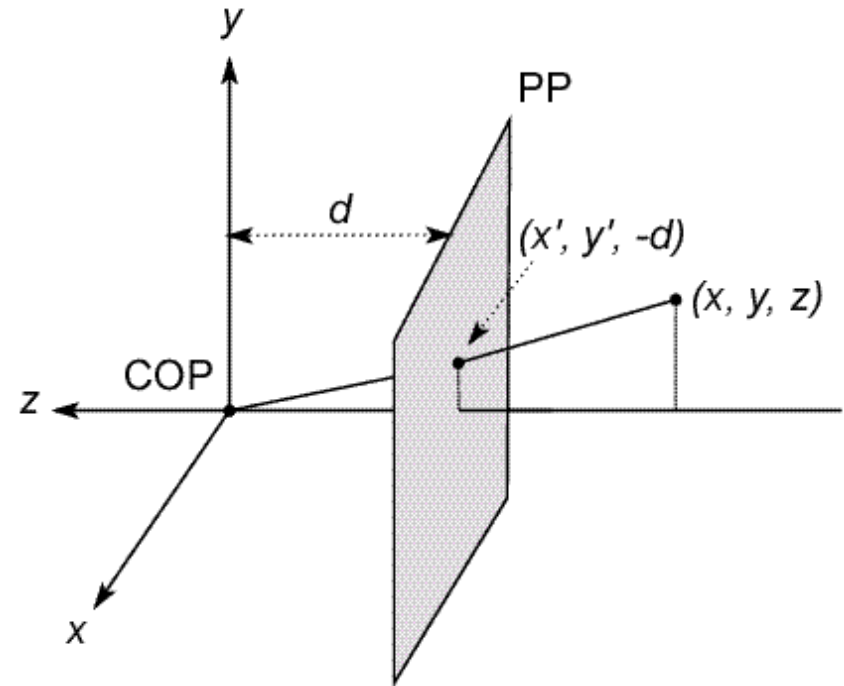
$$U = -X * \frac{f}{Z}$$

$$V = -Y * \frac{f}{Z}$$

What is the effect if f and Z are equal?

Modeling projection – coordinate system

- We will use the pin-hole model as an approximation
- Put the optical center (Center Of Projection) at the origin
- STANDARD (x, y) COORDINATE SYSTEM
- Put the image plane (Projection Plane) in front of the COP (why?)
- The camera looks down the negative z axis

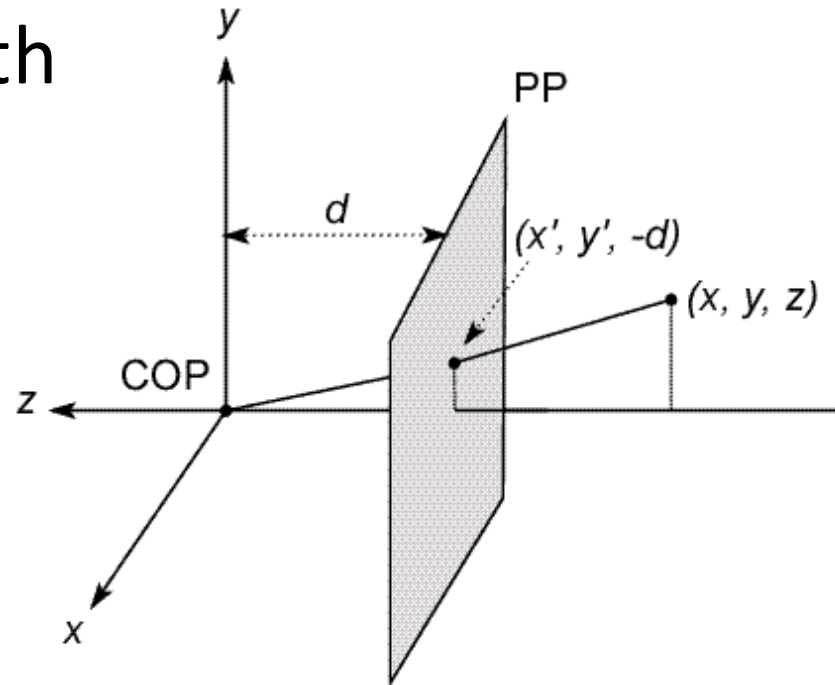


Modeling projection – coordinate system

Projection equations

- Compute intersection with Perspective Projection of ray from (x,y,z) to COP
- Derived using similar

$$(X, Y, Z) \rightarrow \left(-d \frac{X}{Z}, -d \frac{Y}{Z}, -d \right)$$



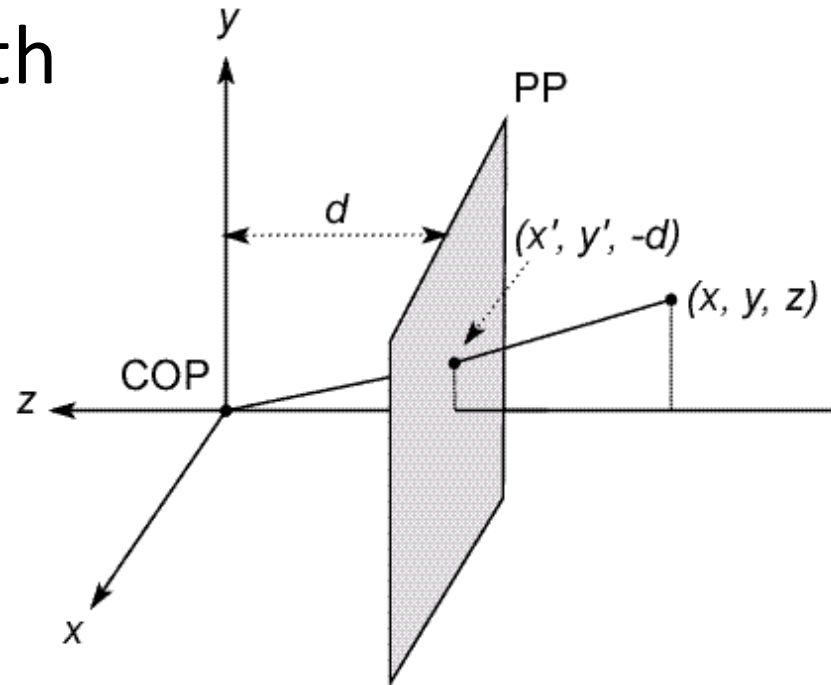
- (assumes normal Z negative – we'll change later)

Modeling projection – coordinate system

Projection equations

- Compute intersection with Perspective Projection of ray from (x,y,z) to COP
- Derived using similar

$$(X, Y, Z) \rightarrow \left(-d \frac{X}{Z}, -d \frac{Y}{Z}, -d \right)$$



- (assumes normal Z negative – we'll change later)

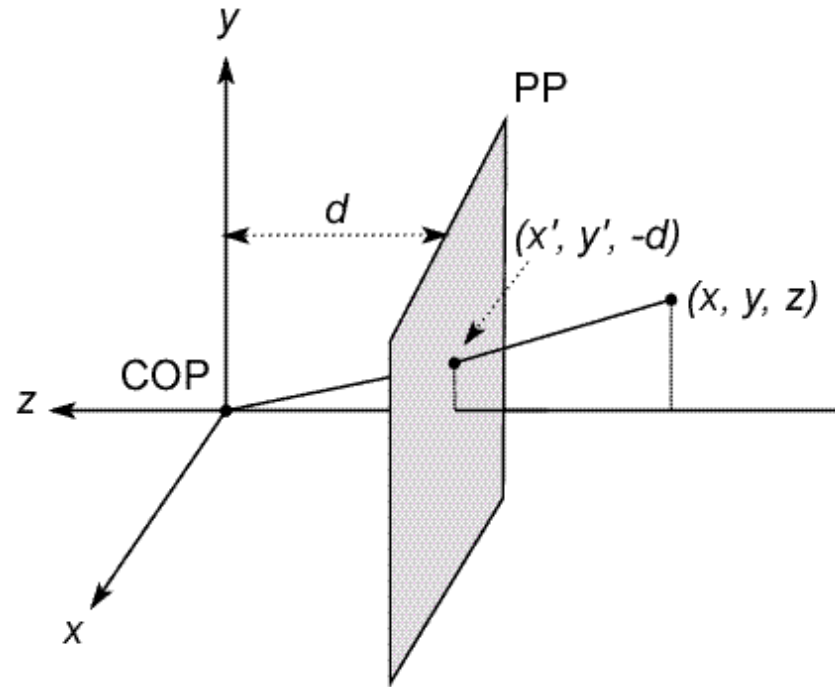
Modeling projection – coordinate system

Projection equations

$$(X, Y, Z) \rightarrow \left(-d\frac{X}{Z}, -d\frac{Y}{Z}, -d \right)$$

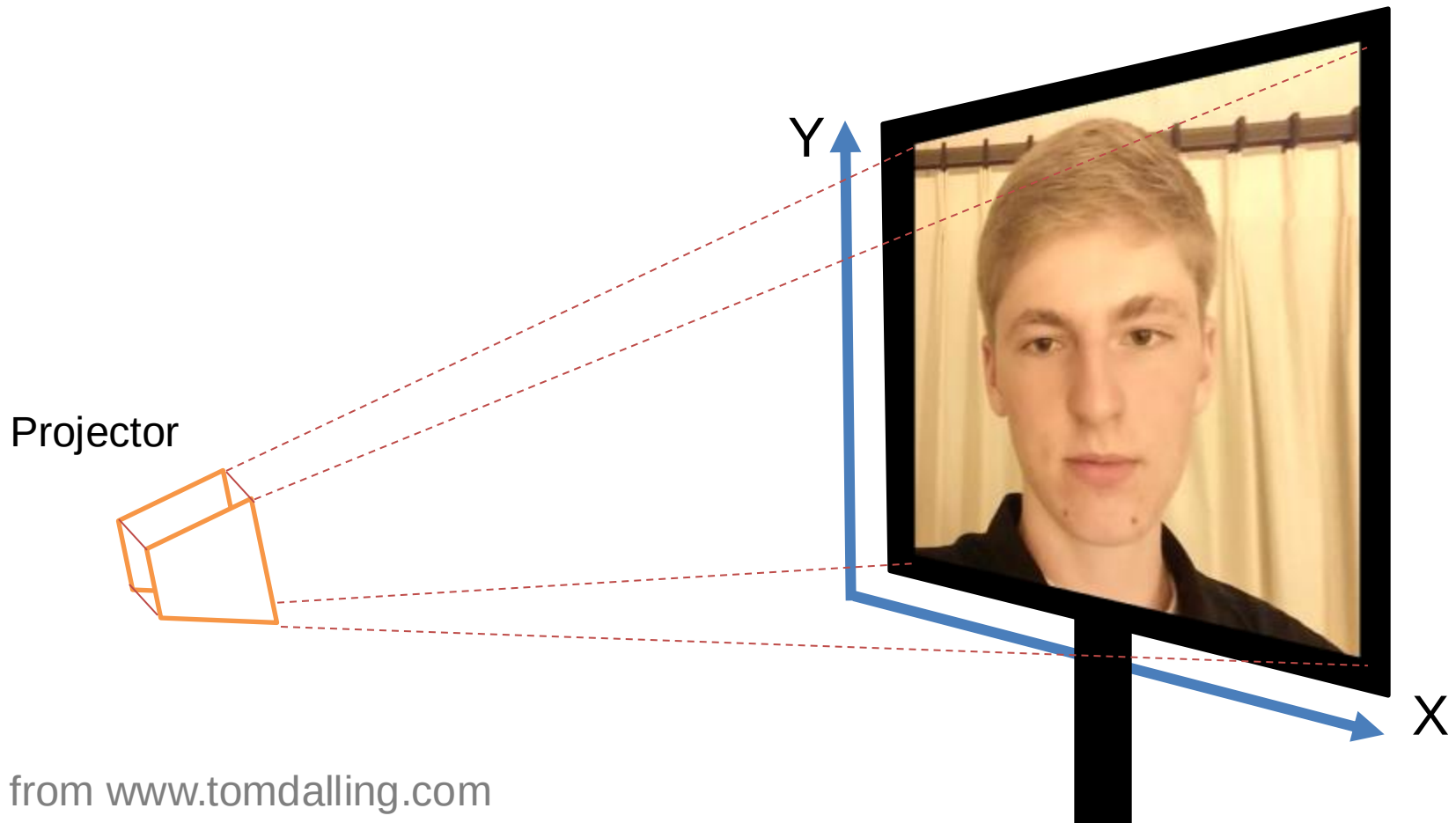
We get the projection by throwing out the last coordinate:

$$(x', y') = \left(-d\frac{X}{Z}, -d\frac{Y}{Z} \right)$$



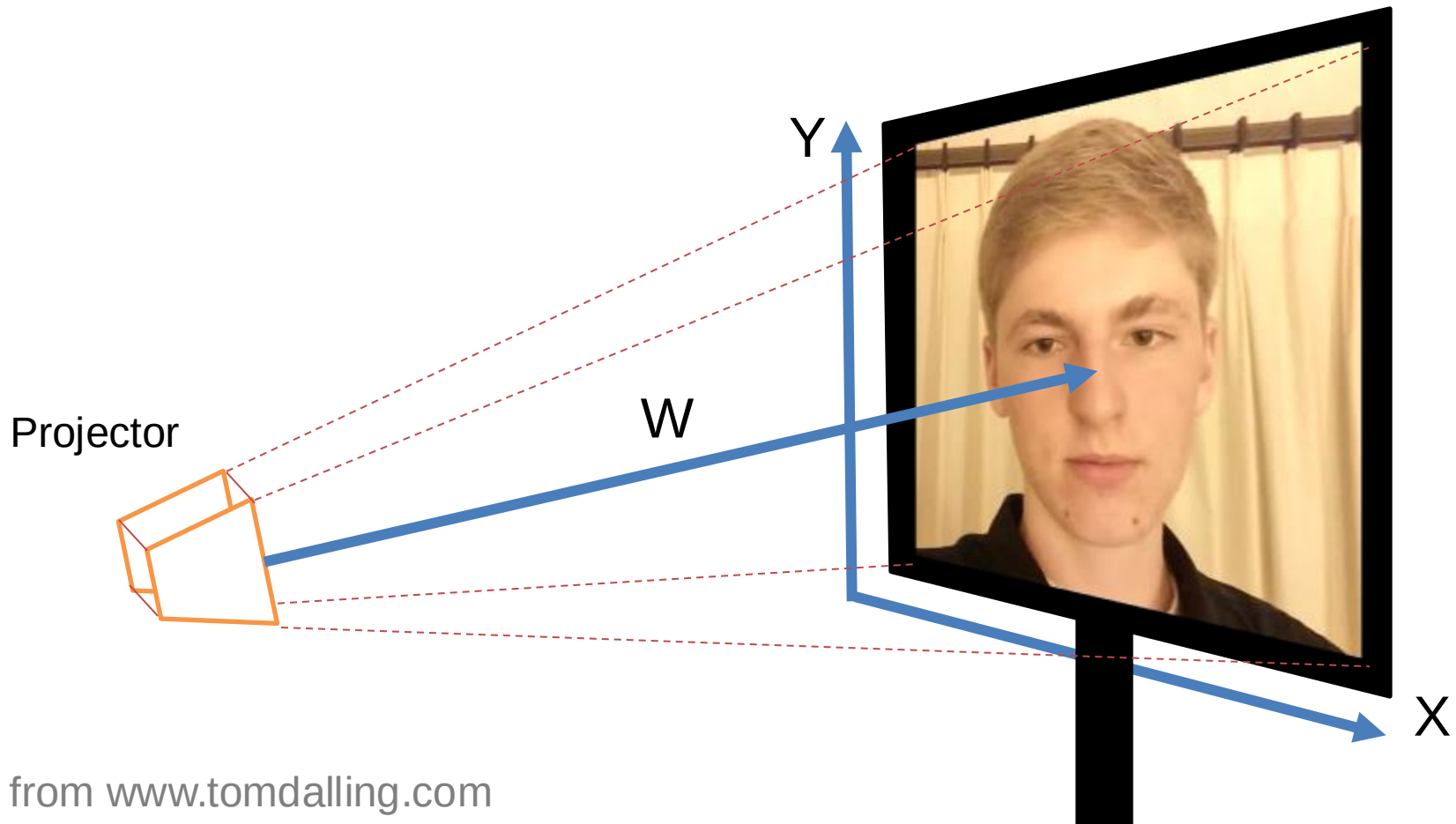
Projective geometry

- 2D point in cartesian = (x,y) coordinates
- 2D point in projective = (x,y,w) coordinates



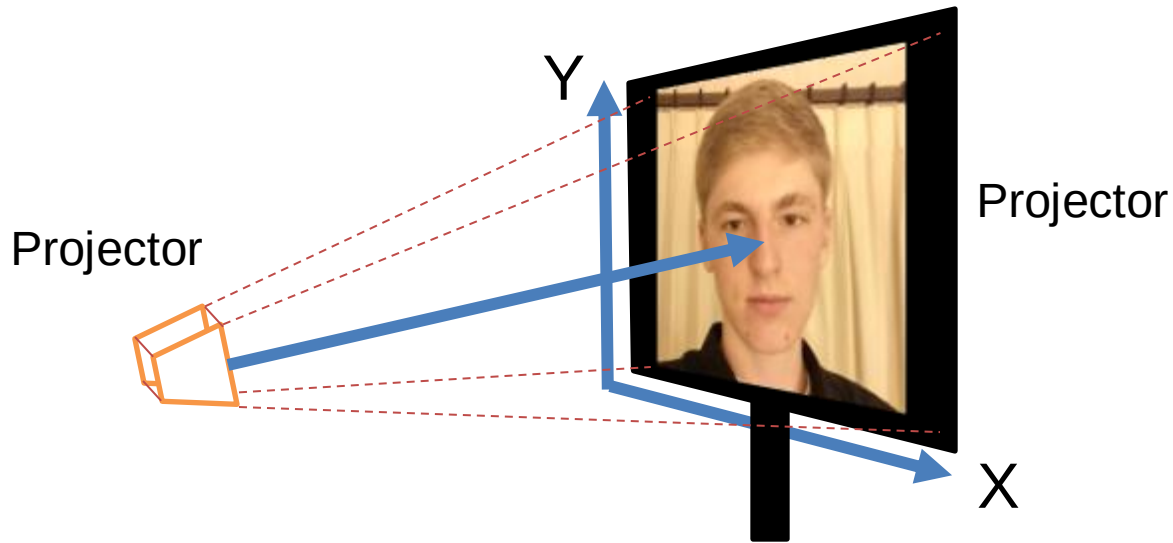
Projective geometry

- 2D point in cartesian = (x,y) coordinates
- 2D point in projective = (x,y,w) coordinates

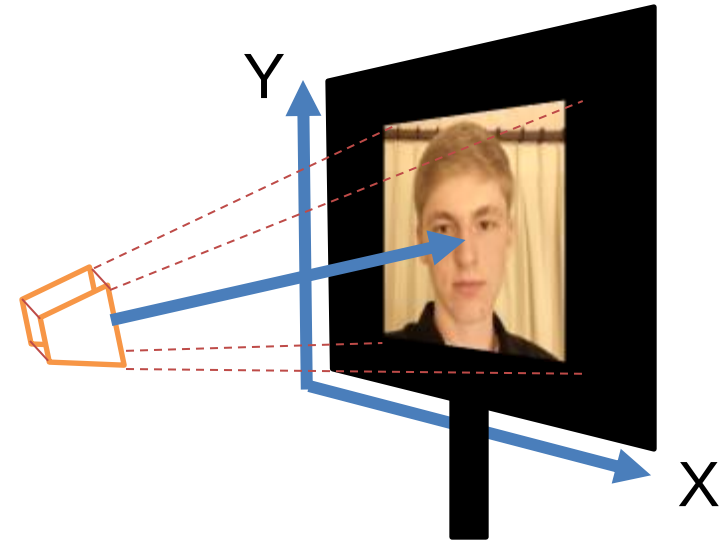


Varying w

W_1



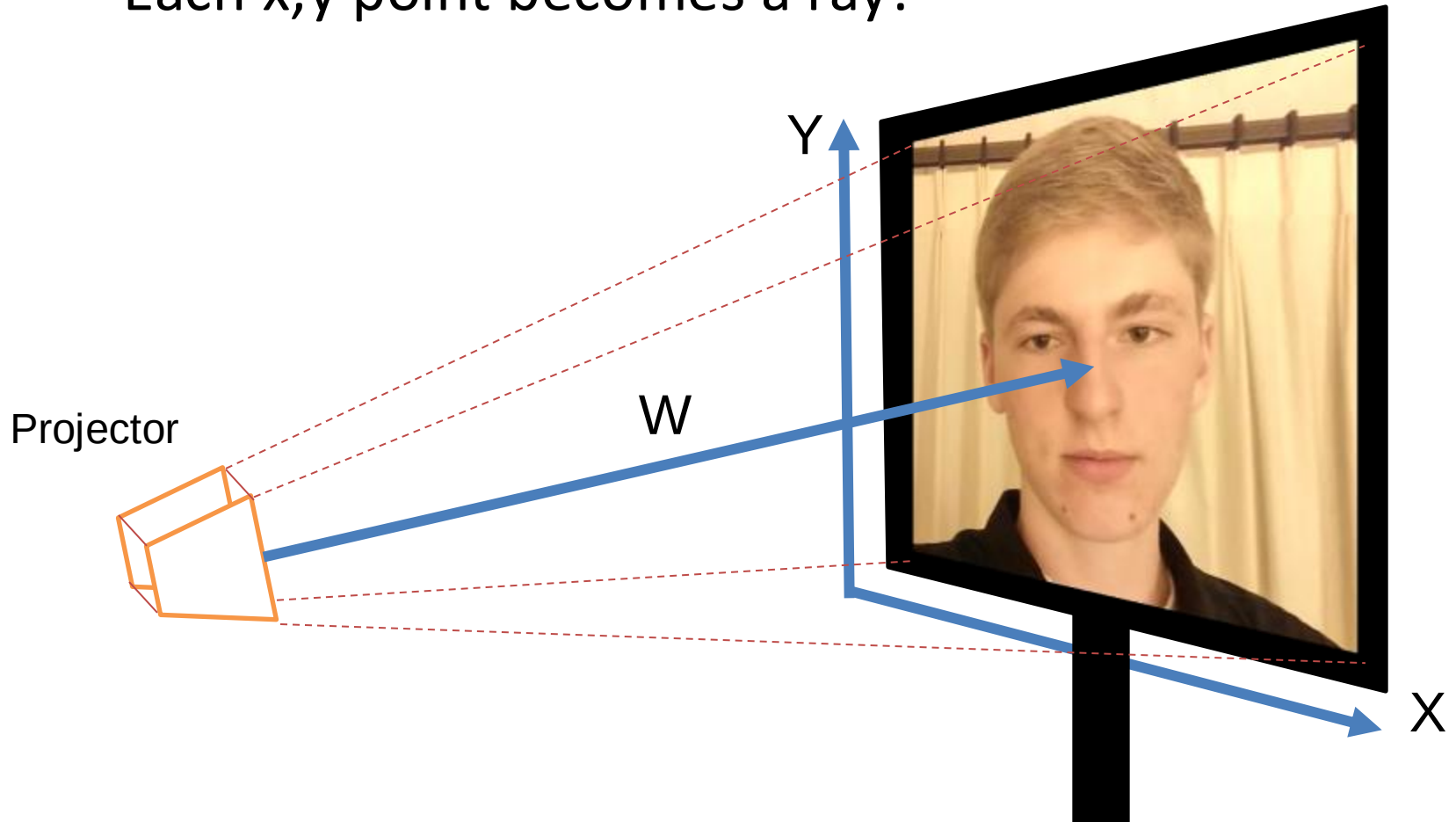
$W_2 < W_1$



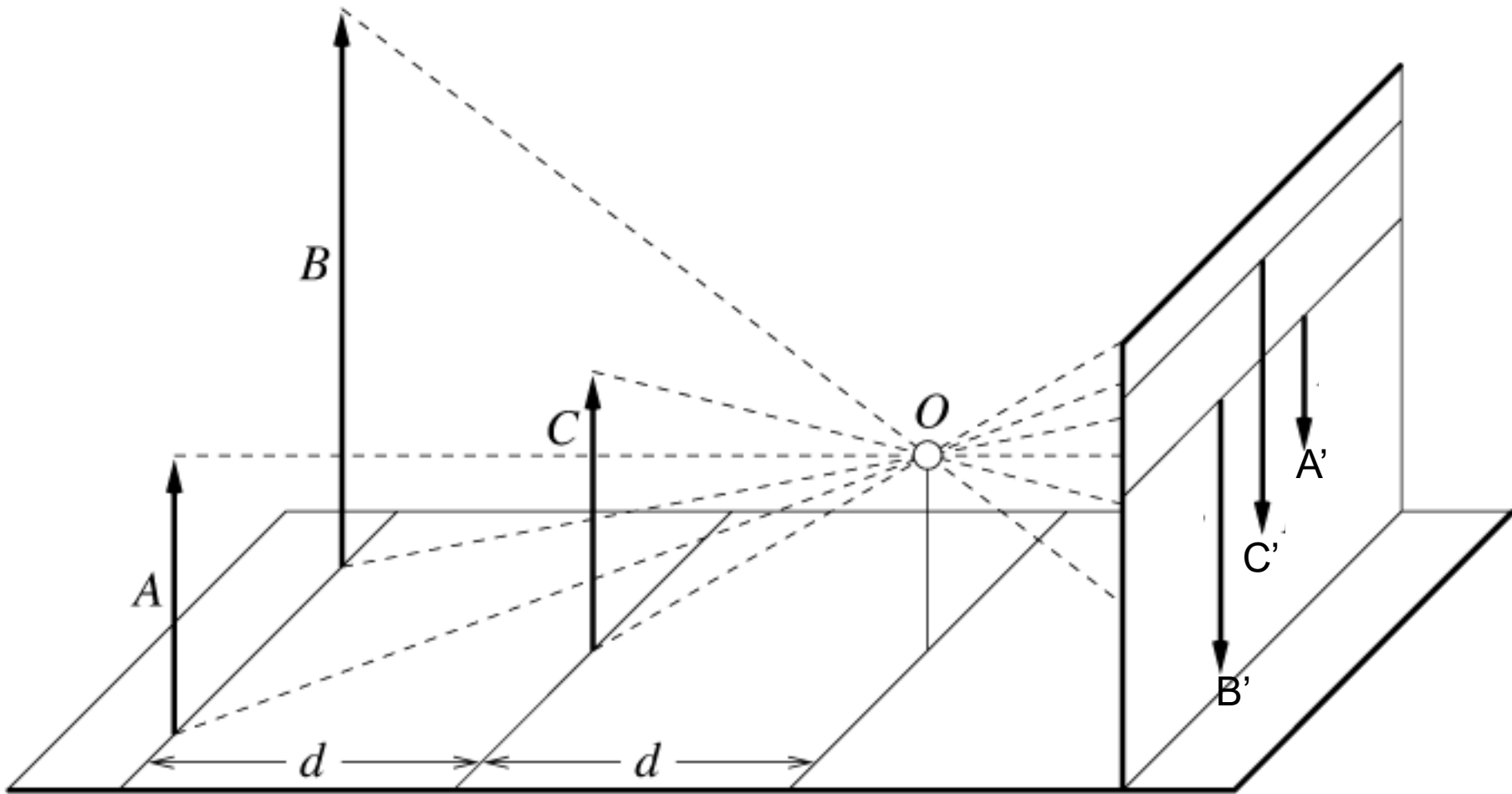
Projected image becomes smaller.

Projective geometry

- 2D point in projective = (x,y,w) coordinates
 - w defines the scale of the projected image.
 - Each x,y point becomes a ray!

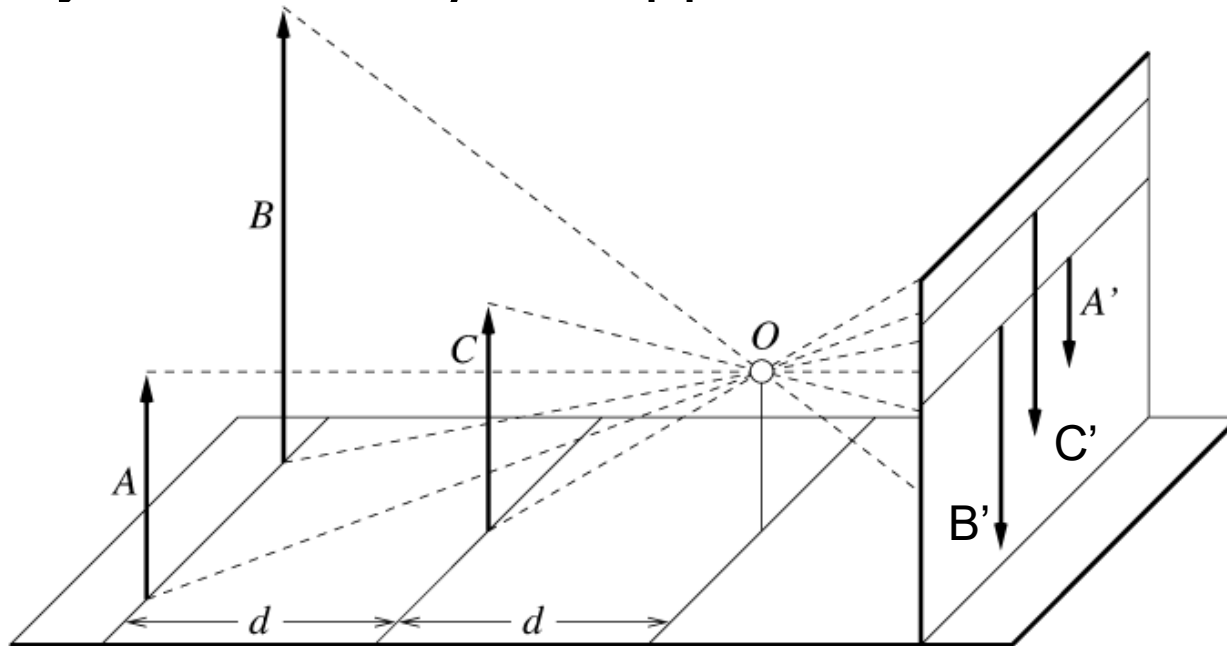


Length and area are not preserved



Projective geometry

- In 3D, point (x,y,z) becomes (x,y,z,w)
- Perspective is w varying with z :
 - Objects far away appear smaller



- Can we use Linear algebra --- is this linear?
- Nope: division by z is non-linear

Homogeneous coordinates

$$(x', y') = \left(-d \frac{X}{Z}, -d \frac{Y}{Z} \right)$$

Is this a linear transformation?

- No – division by the (not constant) Z is non-linear

- Trick: add one more coordinate

$$(x, y) \Rightarrow \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

homogeneous image
(2D) coordinates

$$(x, y, z) \Rightarrow \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$

homogeneous scene
(3D) coordinates

Homogeneous Coordinates (2D)

Trick: add a dimension!

This also clears up lots of nasty special cases

Physical
Point

$$\begin{bmatrix} x \\ y \end{bmatrix}$$



Concat
 $w=1$

Homogeneous
Point

$$\begin{bmatrix} u \\ v \\ w \end{bmatrix}$$



Divide
by w

Physical
Point

$$\begin{bmatrix} u/w \\ v/w \end{bmatrix}$$

What if $w = 0$?

Homogeneous coordinates

- Converting from homogeneous coordinates:

For 2D homogeneous coordinates:

$$\begin{bmatrix} x \\ y \\ w \end{bmatrix} \Rightarrow \left(\frac{x}{w}, \frac{y}{w} \right)$$

For 3D homogeneous coordinates:

$$\begin{bmatrix} x \\ y \\ z \\ w \end{bmatrix} \Rightarrow \left(\frac{x}{w}, \frac{y}{w}, \frac{z}{w} \right)$$

(this makes homogeneous coordinates invariant under scale)

(x, y, w) is equivalent to (kx, ky, kw) for any $k \neq 0$

Homogeneous Coordinates

Triple /
Equivalent

$$\begin{bmatrix} u \\ v \\ w \end{bmatrix} \equiv \begin{bmatrix} u' \\ v' \\ w' \end{bmatrix}$$

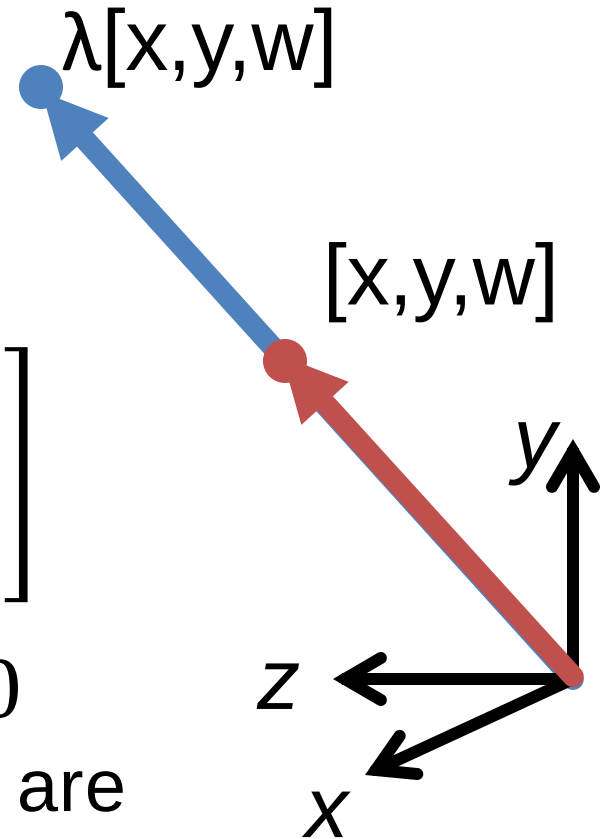
$\leftrightarrow$

Double /
Equals

$$\begin{bmatrix} u \\ v \\ w \end{bmatrix} = \lambda \begin{bmatrix} u' \\ v' \\ w' \end{bmatrix}$$

$\lambda \neq 0$

Two homogeneous coordinates are **equivalent** if they are proportional to each other. **Not = !**



Homogeneous coordinates

Scale invariance in projection space

$$k \begin{bmatrix} x \\ y \\ w \end{bmatrix} = \begin{bmatrix} kx \\ ky \\ kw \end{bmatrix} \Rightarrow \begin{bmatrix} \frac{kx}{kw} \\ \frac{ky}{kw} \\ \frac{kw}{kw} \end{bmatrix} = \begin{bmatrix} \frac{x}{w} \\ \frac{y}{w} \\ 1 \end{bmatrix}$$

Homogeneous Coordinates Cartesian Coordinates

E.G., we can uniformly scale the projective space, and it will still produce the same image -> *scale ambiguity*

Homogeneous coordinates

Converting *to* homogeneous coordinates

$$(x, y) \Rightarrow \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

2D (image) coordinates

$$(x, y, z) \Rightarrow \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$

3D (scene) coordinates

Converting *from* homogeneous coordinates

$$\begin{bmatrix} x \\ y \\ w \end{bmatrix} \Rightarrow (x/w, y/w)$$

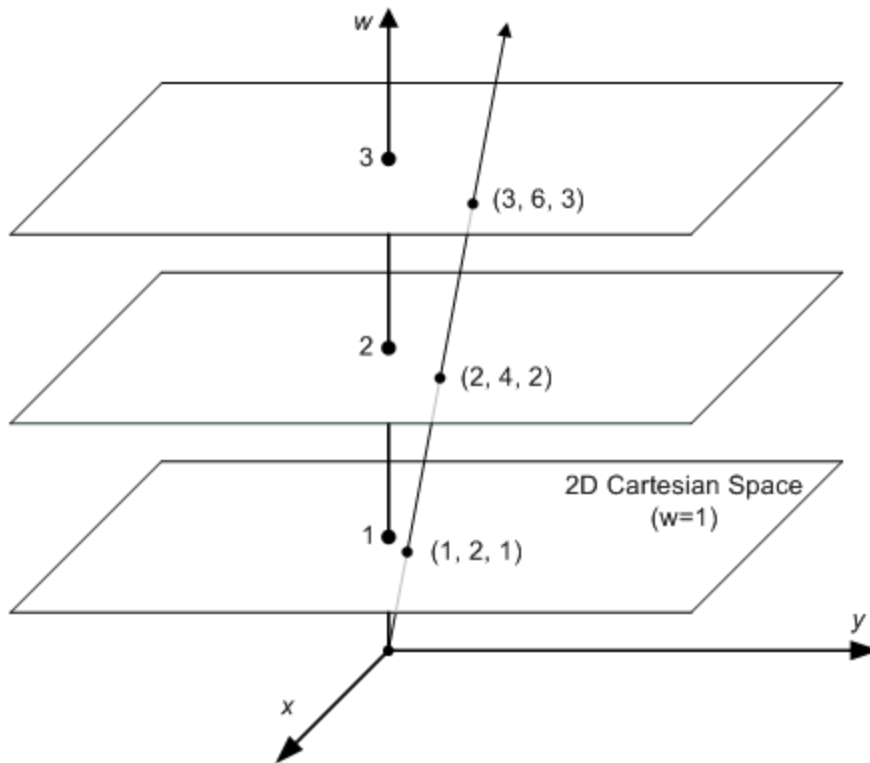
2D (image) coordinates

$$\begin{bmatrix} x \\ y \\ z \\ w \end{bmatrix} \Rightarrow (x/w, y/w, z/w)$$

3D (scene) coordinates

Homogeneous coordinates

- Projective
- Point becomes a line



To homogeneous

$$(x, y) \Rightarrow \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

From homogeneous

$$\begin{bmatrix} x \\ y \\ w \end{bmatrix} \Rightarrow (x/w, y/w)$$

Benefits of Homogeneous Coords

General equation of 2D line:

$$ax + by + c = 0$$

In Homogeneous Coordinates,
line is represented as:

$$L = \begin{bmatrix} a \\ b \\ c \end{bmatrix}$$

A point $P = \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$ lies on the line if:

$$L^T P = ax + by + c = 0$$

Benefits of Homogeneous Coords

Homogeneous Coordinates in $\mathbb{P}^2$:

- A **point** is represented by a **3-dimensional column vector**:

$$\mathbf{P} = \begin{bmatrix} x \\ y \\ w \end{bmatrix} \in \mathbb{P}^2$$

- A **line** is represented by a **3-dimensional column vector**:

$$\mathbf{L} = \begin{bmatrix} a \\ b \\ c \end{bmatrix} \in \mathbb{P}^2$$

- Incidence relation (Point $\mathbf{P}$ lies on Line $\mathbf{L}$):

$$\mathbf{L}^\top \mathbf{P} = 0$$

Both live in **3-dimensional vector space**

$$ax + by + c = 0$$

Benefits of Homogeneous Coords

Given two lines:

$$\mathbf{L}_1 = \begin{bmatrix} a_1 \\ b_1 \\ c_1 \end{bmatrix}, \quad \mathbf{L}_2 = \begin{bmatrix} a_2 \\ b_2 \\ c_2 \end{bmatrix}$$

Their intersection point is found by computing the **cross product**:

$$\mathbf{P} = \mathbf{L}_1 \times \mathbf{L}_2$$

Parallel Lines Condition:

Two lines are **parallel** if their **direction vectors** are proportional.

In homogeneous coordinates, this condition is:

$$a_1 b_2 - a_2 b_1 = 0$$

Perpendicular Lines Condition:

Two lines are **perpendicular** if their direction vectors are orthogonal:

$$a_1 a_2 + b_1 b_2 = 0$$

Benefits of Homogeneous Coords

Given two points:

$$\mathbf{P}_1 = \begin{bmatrix} x_1 \\ y_1 \\ 1 \end{bmatrix}, \quad \mathbf{P}_2 = \begin{bmatrix} x_2 \\ y_2 \\ 1 \end{bmatrix}$$

The line passing through them is:

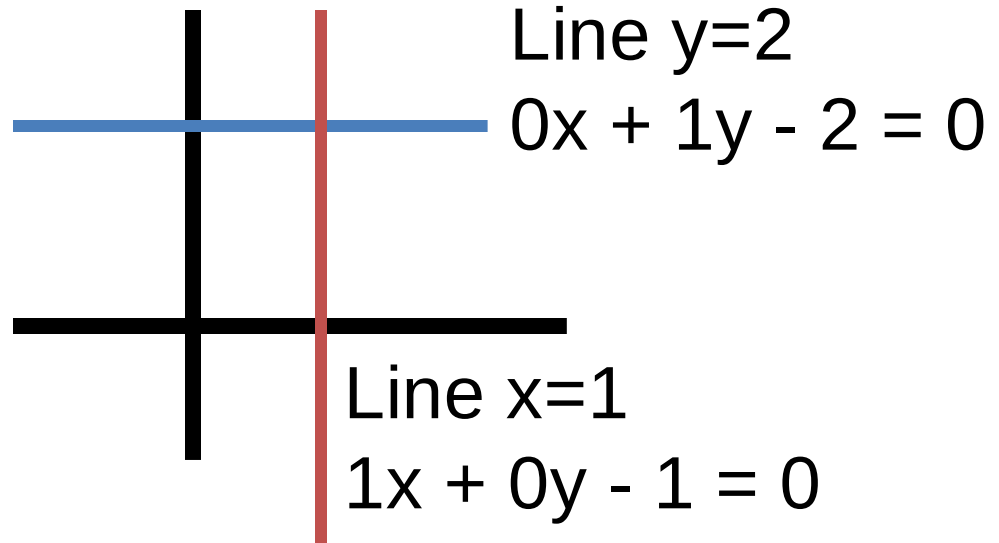
$$\mathbf{L} = \mathbf{P}_1 \times \mathbf{P}_2$$

Recap: Homogeneous Coords

$$(a_1, b_1, c_1) = (0, 1, -2)$$

$$(a_2, b_2, c_2) = (1, 0, -1)$$

Intersection: $l_1 \times l_2$

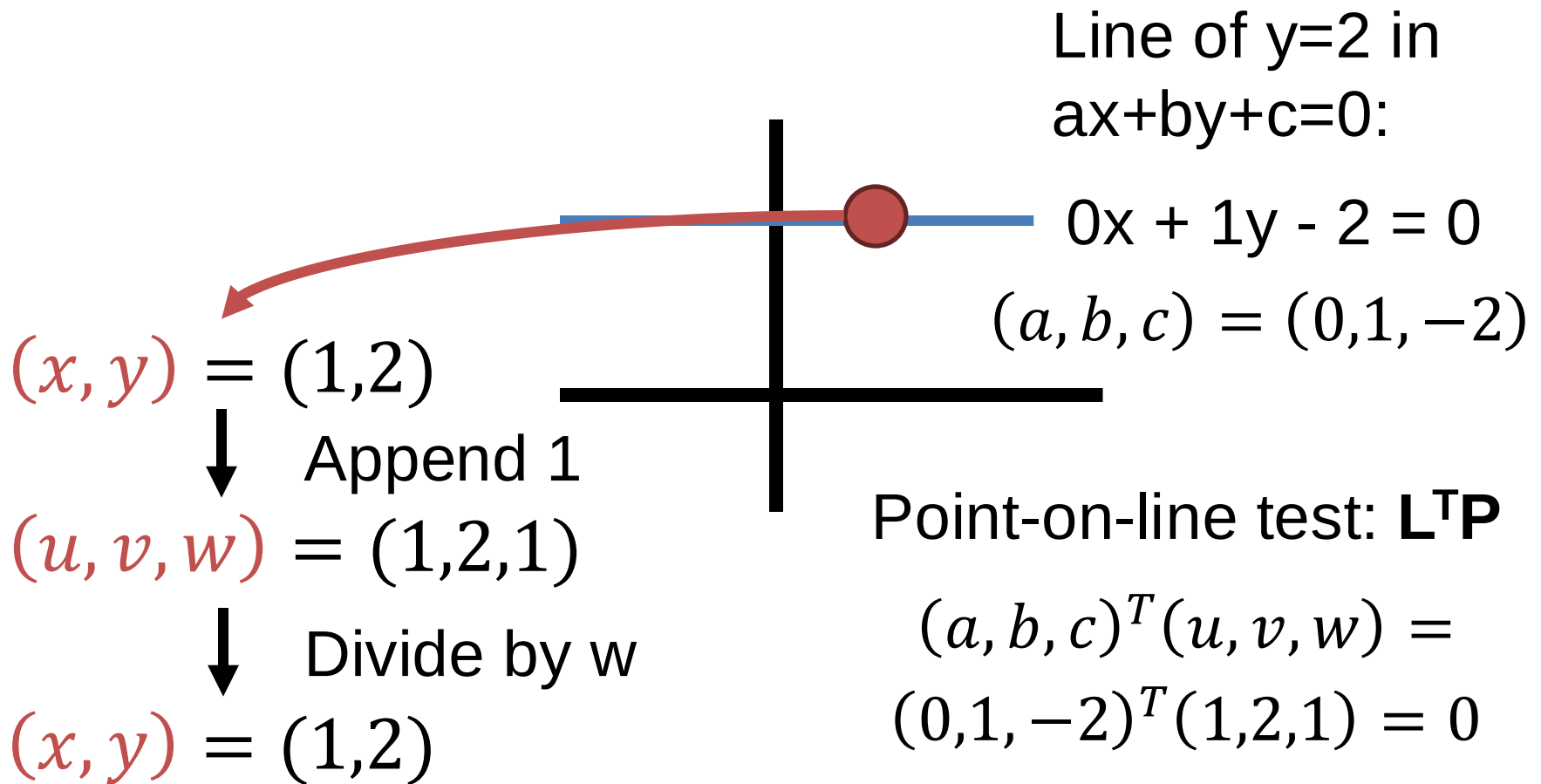


$$[0, 1, -2] \times [1, 0, -1] = [-1, -2, -1]$$

Converting back (divide by -1)

$$(1, 2)$$

Recap: Homogeneous Coords



Benefits of Homogeneous Coords



Intersection of $y=2$, $y=1$

$$[0,1,-2] \times [0,1,-1] = [1,0,0]$$

Interpretation: This is a **point at infinity** in the **x-direction**.

It makes sense because parallel lines intersect at infinity in projective geometry.

Does the intersection point lie on $y=3$?

$$[0,1,-3]^T [1,0,0] = 0$$

Yes, it **does lie** on the line $y=3$

intuitively, because this **point at infinity** represents the **direction** shared by **all parallel lines**

Benefits of Homogeneous Coords

Translation is now linear / matrix-multiply

$$\text{If } w = 1 \quad \begin{bmatrix} u' \\ v' \\ w' \end{bmatrix} = \begin{bmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = \begin{bmatrix} u + t_x \\ v + t_y \\ 1 \end{bmatrix}$$

$$\text{Generically} \quad \begin{bmatrix} u' \\ v' \\ w' \end{bmatrix} = \begin{bmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} u \\ v \\ w \end{bmatrix} = \begin{bmatrix} u + wt_x \\ v + wt_y \\ w \end{bmatrix}$$

Rigid body transforms (rot + trans) now linear

$$\begin{bmatrix} u' \\ v' \\ w' \end{bmatrix} = \begin{bmatrix} r_{11} & r_{12} & t_x \\ r_{21} & r_{22} & t_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} u \\ v \\ w \end{bmatrix}$$

3D Homogeneous Coordinates

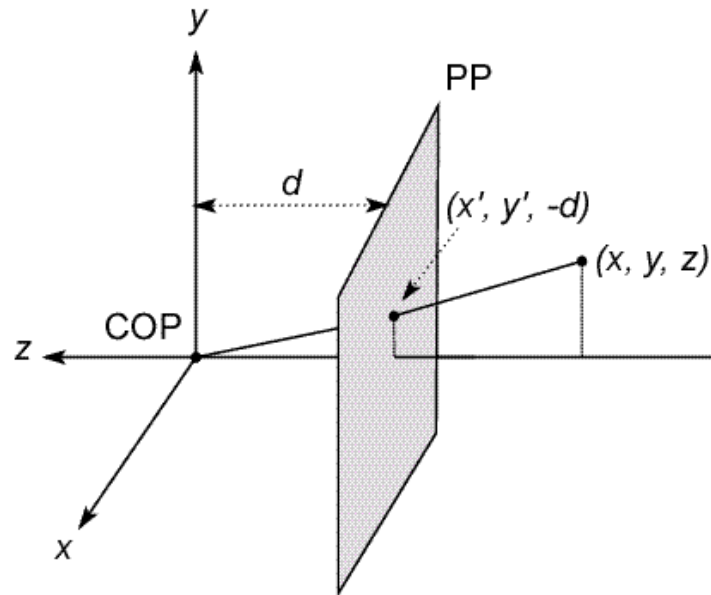
Same story: add a coordinate, things are equivalent if they're proportional

$$\begin{bmatrix} x \\ y \\ z \end{bmatrix} \longrightarrow \begin{bmatrix} u \\ v \\ w \\ t \end{bmatrix} \longrightarrow \begin{bmatrix} u/t \\ v/t \\ w/t \end{bmatrix}$$

Basic geometry in homogeneous coordinates

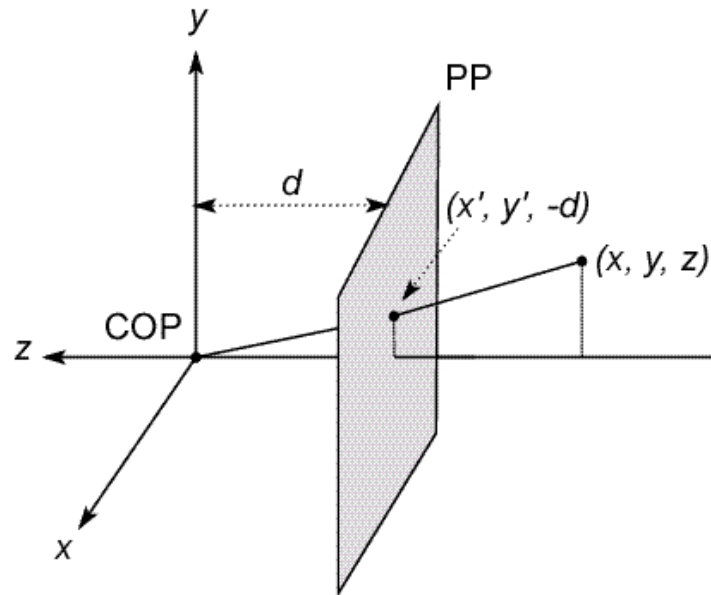
- Line equation: $ax + by + c = 0$ $line_i = \begin{bmatrix} a_i \\ b_i \\ c_i \end{bmatrix}$
- Append 1 to pixel coordinate to get homogeneous coordinate $p_i = \begin{bmatrix} u_i \\ v_i \\ 1 \end{bmatrix}$
- Line given by cross product of two points $line_{ij} = p_i \times p_j$
- Intersection of two lines given by cross product of the lines $q_{ij} = line_i \times line_j$

Modeling projection



- The coordinate system
 - We will use the pinhole model as an approximation
 - Put the optical center (**C**enter **O**f **P**rojection) at the origin
 - Put the image plane (**P**rojection **P**lane) *in front* of the COP
 - Why?
 - The camera looks down the *negative* z axis
 - we need this if we want right-handed-coordinates

Modeling projection



- Projection equations

- Compute intersection with PP of ray from (x,y,z) to COP
- Derived using similar triangles (on board)

$$(x, y, z) \rightarrow \left(-d\frac{x}{z}, -d\frac{y}{z}, -d\right)$$

- We get the projection by throwing out the last coordinate:

$$(x, y, z) \rightarrow \left(-d\frac{x}{z}, -d\frac{y}{z}\right)$$

Modeling projection

- Is this a linear transformation?
 - no—division by z is nonlinear

Homogeneous coordinates to the rescue!

$$(x, y) \Rightarrow \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

homogeneous image
coordinates

$$(x, y, z) \Rightarrow \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$

homogeneous scene
coordinates

Converting *from* homogeneous coordinates

$$\begin{bmatrix} x \\ y \\ w \end{bmatrix} \Rightarrow (x/w, y/w)$$

$$\begin{bmatrix} x \\ y \\ z \\ w \end{bmatrix} \Rightarrow (x/w, y/w, z/w)$$

Perspective Projection

Projection is a matrix multiply using homogeneous coordinates:

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & -1/d & 0 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix} = \begin{bmatrix} x \\ y \\ -z/d \end{bmatrix} \Rightarrow \left(-d \frac{x}{z}, -d \frac{y}{z} \right)$$

divide by third coordinate

This is known as **perspective projection**

- The matrix is the **projection matrix**

Perspective Projection

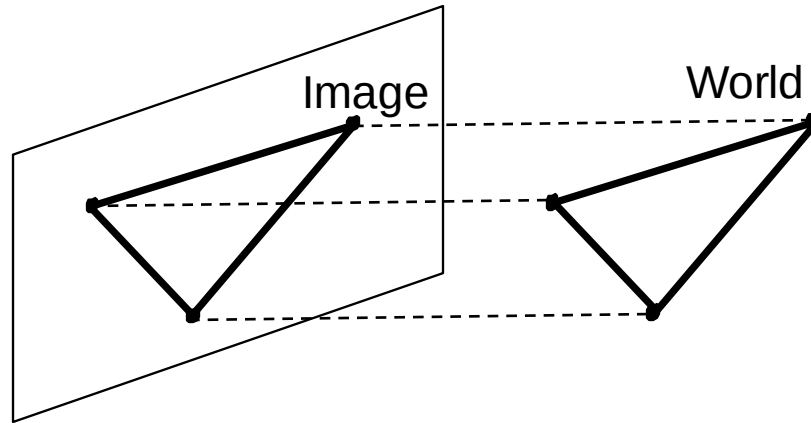
- How does scaling the projection matrix change the transformation?

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & -1/d & 0 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix} = \begin{bmatrix} x \\ y \\ -z/d \end{bmatrix} \Rightarrow \left(-d\frac{x}{z}, -d\frac{y}{z}\right)$$

$$\begin{bmatrix} -d & 0 & 0 & 0 \\ 0 & -d & 0 & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix} = \begin{bmatrix} -dx \\ -dy \\ z \end{bmatrix} \Rightarrow \left(-d\frac{x}{z}, -d\frac{y}{z}\right)$$

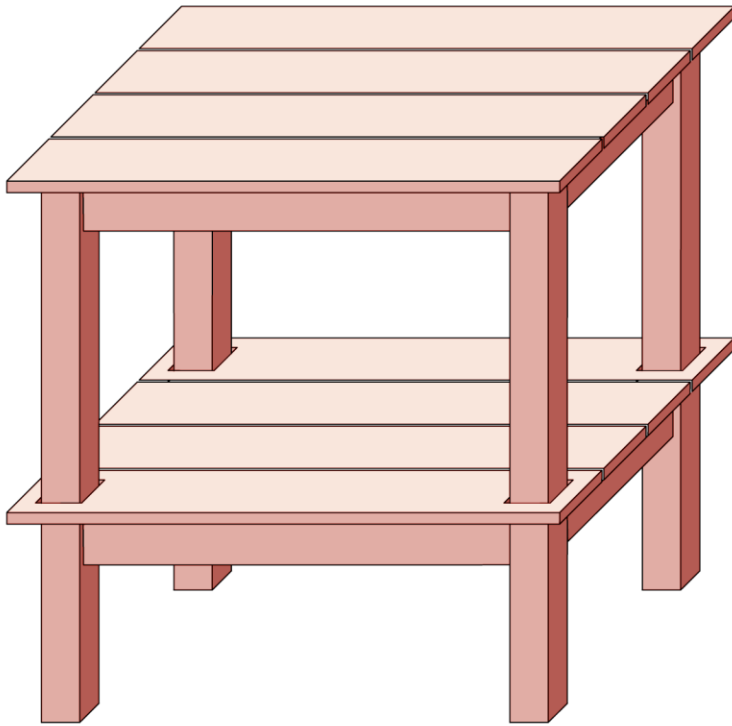
Orthographic projection

- Special case of perspective projection
 - Distance from the COP to the PP is infinite

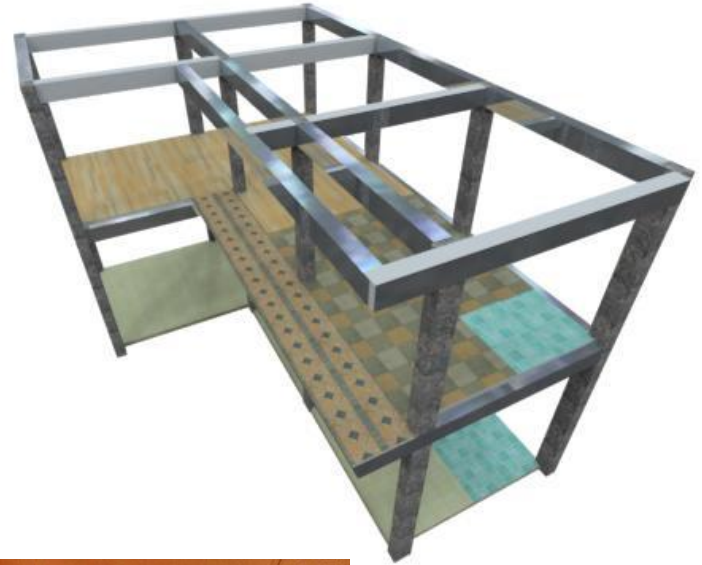


$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix} = \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} \Rightarrow (x, y)$$

Orthographic projection



Perspective projection



Variants of orthographic projection

- Scaled orthographic
 - Also called “weak perspective”

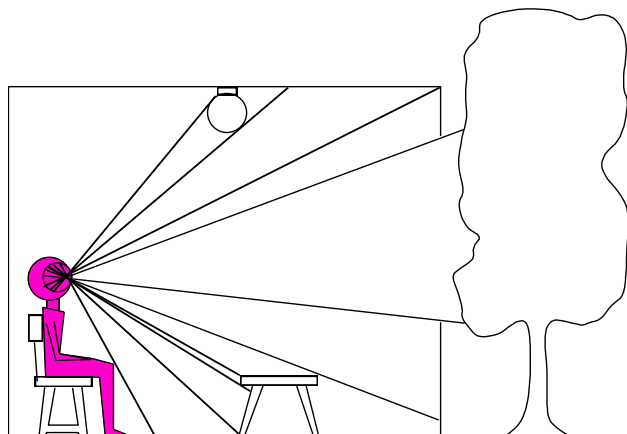
$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1/d \end{bmatrix} \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix} = \begin{bmatrix} x \\ y \\ 1/d \end{bmatrix} \Rightarrow (dx, dy)$$

- Affine projection
 - Also called “paraperspective”

$$\begin{bmatrix} a & b & c & d \\ e & f & g & h \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$

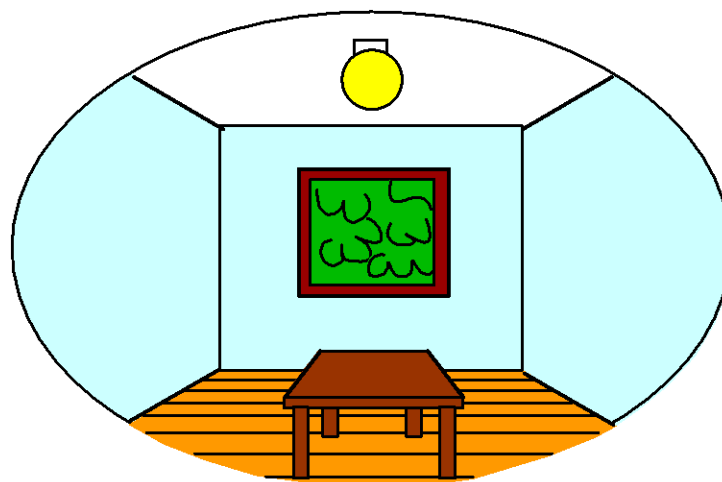
Dimensionality Reduction Machine (3D to 2D)

3D world



Point of observation

2D image



What have we lost?

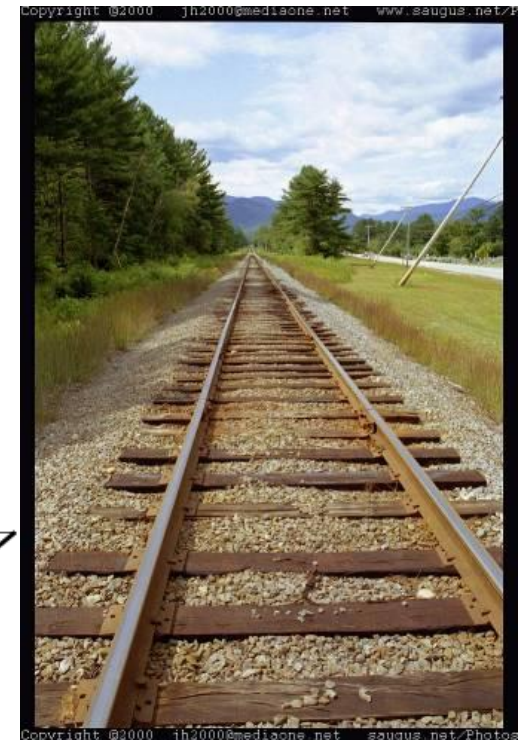
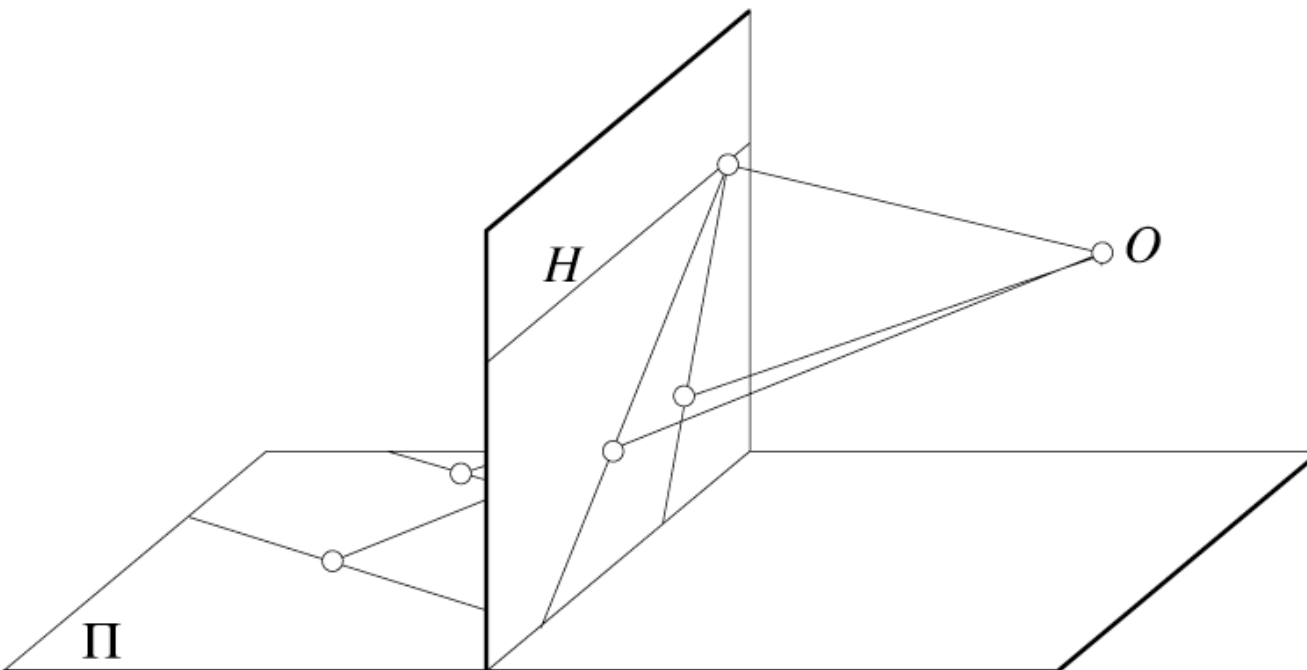
- Angles
- Distances (lengths)

Projection properties

- Many-to-one: any points along same ray map to same point in image
- Points $\rightarrow$ Points
- Lines $\rightarrow$ Lines (collinearity is preserved)
 - But line through focal point projects to a point
- Planes $\rightarrow$ planes (or half-planes)
 - But plane through focal point projects to line

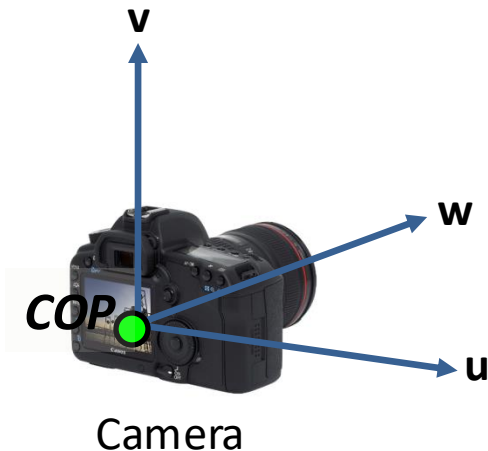
Projection properties

- Parallel lines converge at a vanishing point
 - Each direction in space has its own vanishing point
 - But parallels parallel to the image plane remain parallel



Camera parameters

- How can we model the geometry of a camera?



Two important coordinate systems:

1. *World* coordinate system
2. *Camera* coordinate system



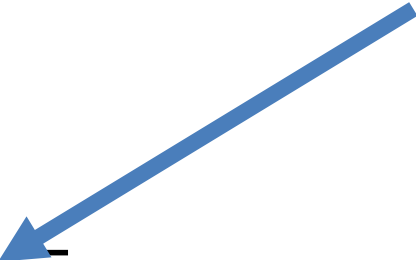
Camera parameters

- To project a point (x, y, z) in world coordinates into a camera
- First transform (x, y, z) into camera coordinates
- Need to know
 - Camera position (in world coordinates)
 - Camera orientation (in world coordinates)
- Then project into the image plane
 - Need to know camera *intrinsics*

Typical Perspective Model

P: 2D homogeneous
point (3D)

$\mathbf{P} \equiv$



X: 3d homogeneous
point (4D)

$\mathbf{X}_{4 \times 1}$



Typical Perspective Model

R: rotation between
world system and
camera

t: translation
between world
system and camera

$P \equiv$

$[R_{3 \times 3}$

$t_{3 \times 1}]$

$X_{4 \times 1}$

Typical Perspective Model

f focal length

u_0, v_0 : principal point

It's where the **optical axis** intersects the **image plane**

$$\mathbf{P} \equiv \begin{bmatrix} f & 0 & u_0 \\ 0 & f & v_0 \\ 0 & 0 & 1 \end{bmatrix} \quad [\mathbf{R}_{3 \times 3} \quad \mathbf{t}_{3 \times 1}] \quad \mathbf{X}_{4 \times 1}$$

In an ideal pinhole camera, (u_0, v_0) point is **exactly** at the center of the image sensor.

Geometric Camera Calibration

- Composed of two calibrations
 1. World co-ordinate to Camera's 3D coordinate
 - Extrinsic Parameters
 2. From 3D coordinate to 2D image plane
 - Intrinsic Parameters

Typical Perspective Model

Intrinsic Matrix K	Extrinsic Matrix $[R, t]$	
$P \equiv \begin{bmatrix} f & 0 & u_0 \\ 0 & f & v_0 \\ 0 & 0 & 1 \end{bmatrix}$	$\begin{bmatrix} R_{3 \times 3} & t_{3 \times 1} \end{bmatrix}$	$X_{4 \times 1}$

$$P \equiv K[R, t]X \equiv M_{3 \times 4}X_{4 \times 1}$$

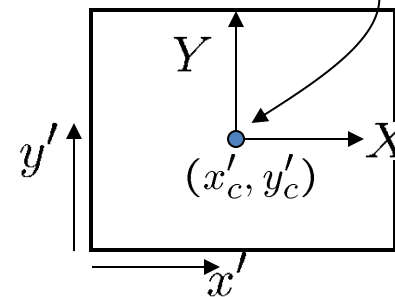
One Slide Summary: Camera parameters

A camera is described by several parameters

- Translation $\mathbf{T}$ of the optical center from the origin of world coords
- Rotation $\mathbf{R}$ of the image plane
- focal length f , principle point (x'_c, y'_c) , pixel size (s_x, s_y)
- blue parameters are called “extrinsics,” red are “intrinsics”

Projection equation

$$\mathbf{X} = \begin{bmatrix} sx \\ sy \\ s \end{bmatrix} = \begin{bmatrix} * & * & * & * \\ * & * & * & * \\ * & * & * & * \end{bmatrix} \begin{bmatrix} X \\ Y \\ Z \\ 1 \end{bmatrix} = \mathbf{\Pi} \mathbf{X}$$



- The projection matrix models the cumulative effect of all parameters
- Useful to decompose into a series of operations

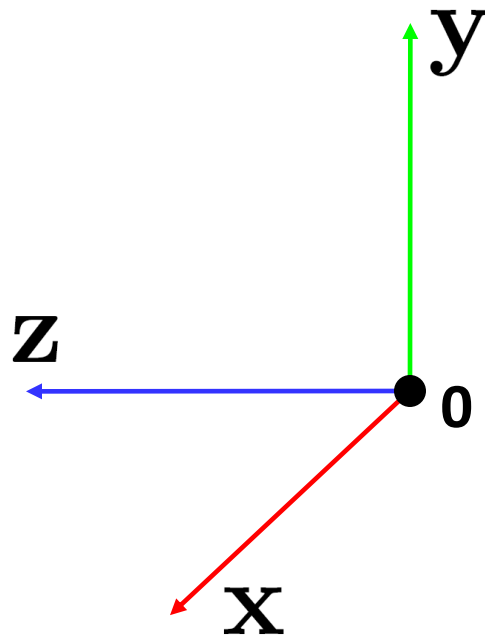
$$\mathbf{\Pi} = \underbrace{\begin{bmatrix} -fs_x & 0 & x'_c \\ 0 & -fs_y & y'_c \\ 0 & 0 & 1 \end{bmatrix}}_{\text{intrinsics}} \underbrace{\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix}}_{\text{projection}} \underbrace{\begin{bmatrix} \mathbf{R}_{3 \times 3} & \mathbf{0}_{3 \times 1} \\ \mathbf{0}_{1 \times 3} & 1 \end{bmatrix}}_{\text{rotation}} \underbrace{\begin{bmatrix} \mathbf{I}_{3 \times 3} & \mathbf{T}_{3 \times 1} \\ \mathbf{0}_{1 \times 3} & 1 \end{bmatrix}}_{\text{translation}}$$

identity matrix

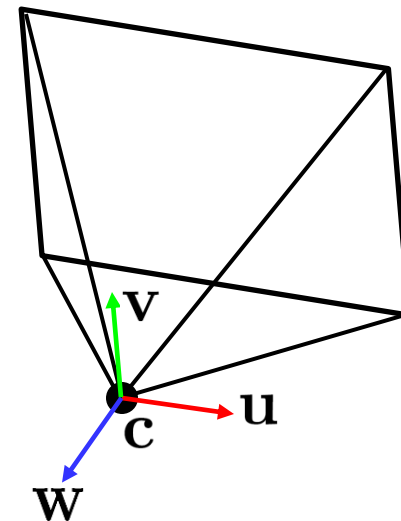
- The definitions of these parameters are **not** completely standardized
 - especially intrinsics—varies from one book to another

Extrinsics

- How do we get the camera to “canonical form”?
 - (Center of projection at the origin)

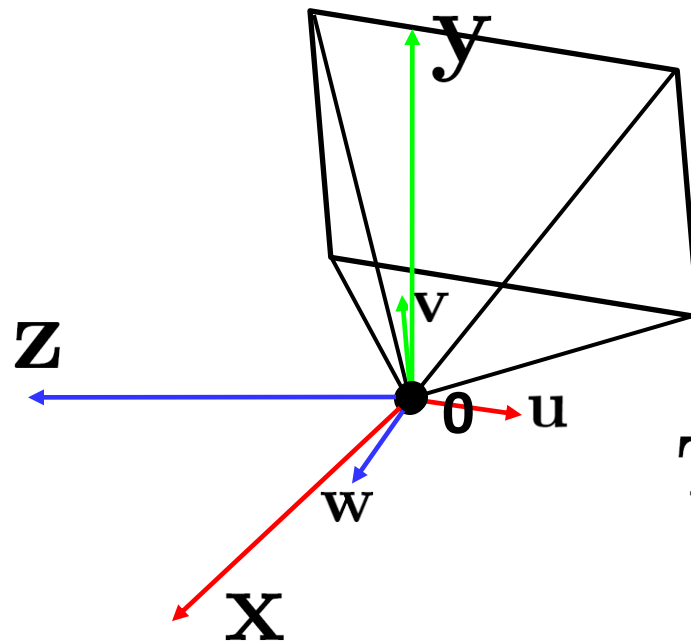


Step 1: Translate by $-c$



Extrinsics

- How do we get the camera to “canonical form”?
 - (Center of projection at the origin, x-axis points right, y-axis points up, z-axis points backwards)



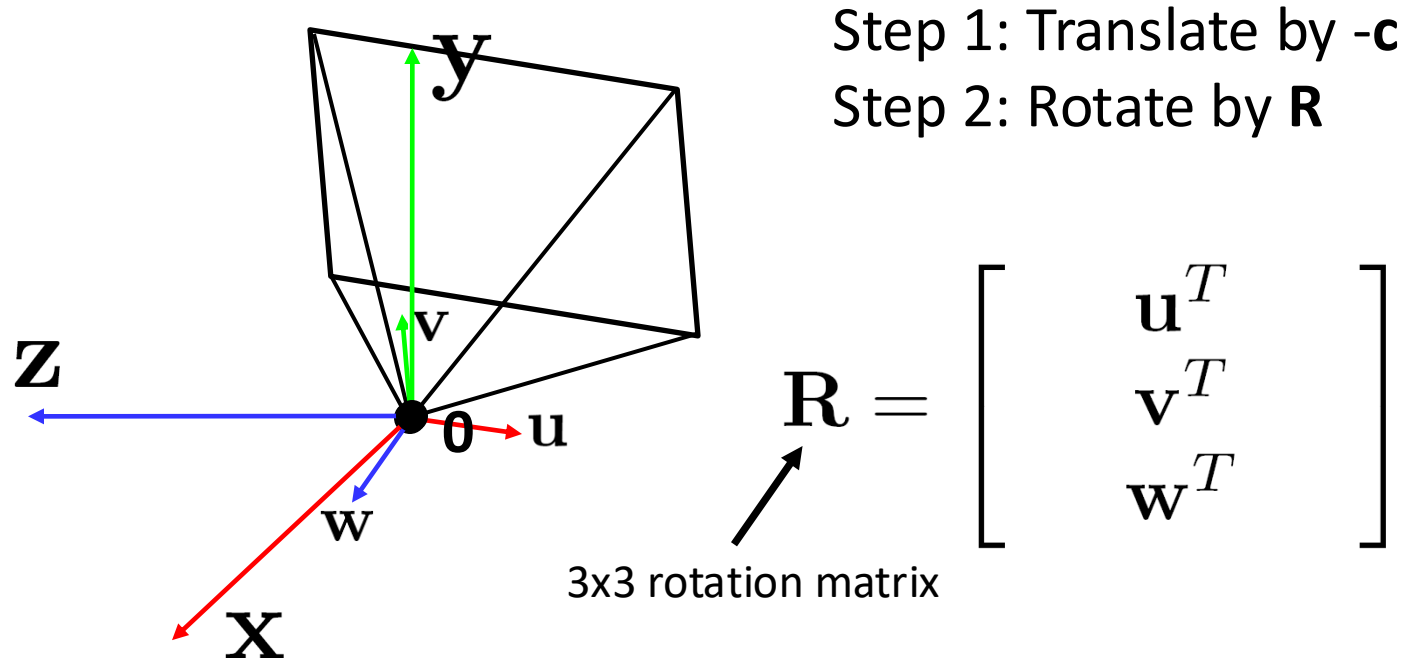
Step 1: Translate by $-\mathbf{c}$

How do we represent translation as a matrix multiplication?

$$\mathbf{T} = \begin{bmatrix} \mathbf{I}_{3 \times 3} & -\mathbf{c} \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

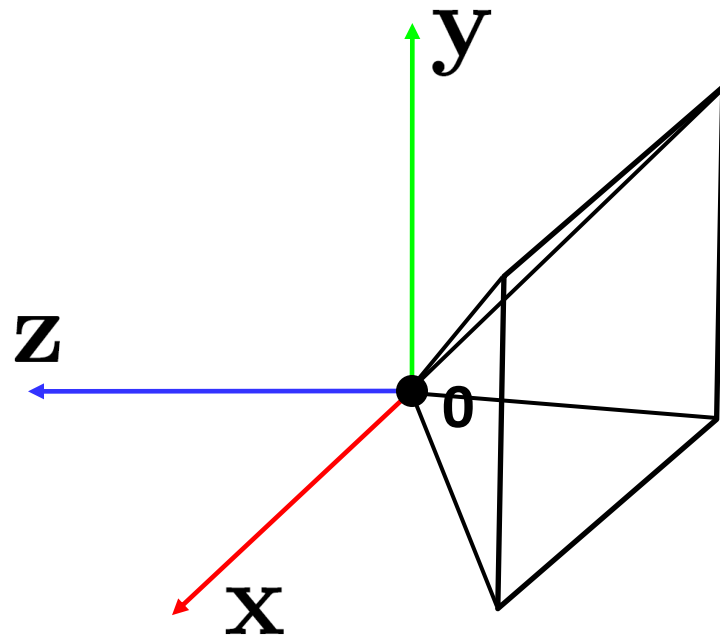
Extrinsics

- How do we get the camera to “canonical form”?
 - (Center of projection at the origin, x-axis points right, y-axis points up, z-axis points backwards)



Extrinsics

- How do we get the camera to “canonical form”?
 - (Center of projection at the origin, x-axis points right, y-axis points up, z-axis points backwards)



Step 1: Translate by $-\mathbf{c}$

Step 2: Rotate by $\mathbf{R}$

$$\mathbf{R} = \begin{bmatrix} \mathbf{u}^T \\ \mathbf{v}^T \\ \mathbf{w}^T \end{bmatrix}$$

Projection Matrix

$$\mathbf{\Pi} = \begin{bmatrix} -fs_x & 0 & x'_c \\ 0 & -fs_y & y'_c \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix} \begin{bmatrix} \mathbf{R}_{3 \times 3} \\ \mathbf{0}_{1 \times 3} \end{bmatrix} \begin{bmatrix} \mathbf{0}_{3 \times 1} \\ 1 \end{bmatrix} \begin{bmatrix} \mathbf{I}_{3 \times 3} & \mathbf{T}_{3 \times 1} \\ \mathbf{0}_{1 \times 3} & 1 \end{bmatrix}$$

Perspective projection

$$\underbrace{\begin{bmatrix} -f & 0 & 0 \\ 0 & -f & 0 \\ 0 & 0 & 1 \end{bmatrix}}_{\mathbf{K}} \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix}$$

K
(intrinsics)

(converts from 3D rays in camera
coordinate system to pixel coordinates)

$$\text{in general, } \mathbf{K} = \begin{bmatrix} -f & s & c_x \\ 0 & -\alpha f & c_y \\ 0 & 0 & 1 \end{bmatrix} \quad \begin{matrix} \text{(upper triangular} \\ \text{matrix)} \end{matrix}$$

α : **aspect ratio** (1 unless pixels are not square)

s : **skew** (0 unless pixels are shaped like rhombi/parallelograms)

(c_x, c_y) : **principal point** ((0,0) unless optical axis doesn't intersect projection plane at origin)

Focal length

- Can think of as “zoom”



24mm



50mm



200mm



800mm



- Also related to field of view

Projection matrix

$$\mathbf{\Pi} = \mathbf{K} \underbrace{\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix}}_{\text{projection}} \underbrace{\begin{bmatrix} \mathbf{R} & \begin{matrix} 0 \\ 0 \\ 0 \end{matrix} \\ 0 & 0 & 0 & 1 \end{bmatrix}}_{\text{rotation}} \underbrace{\begin{bmatrix} \mathbf{I}_{3 \times 3} & -\mathbf{c} \\ 0 & 0 & 0 & 1 \end{bmatrix}}_{\text{translation}}$$

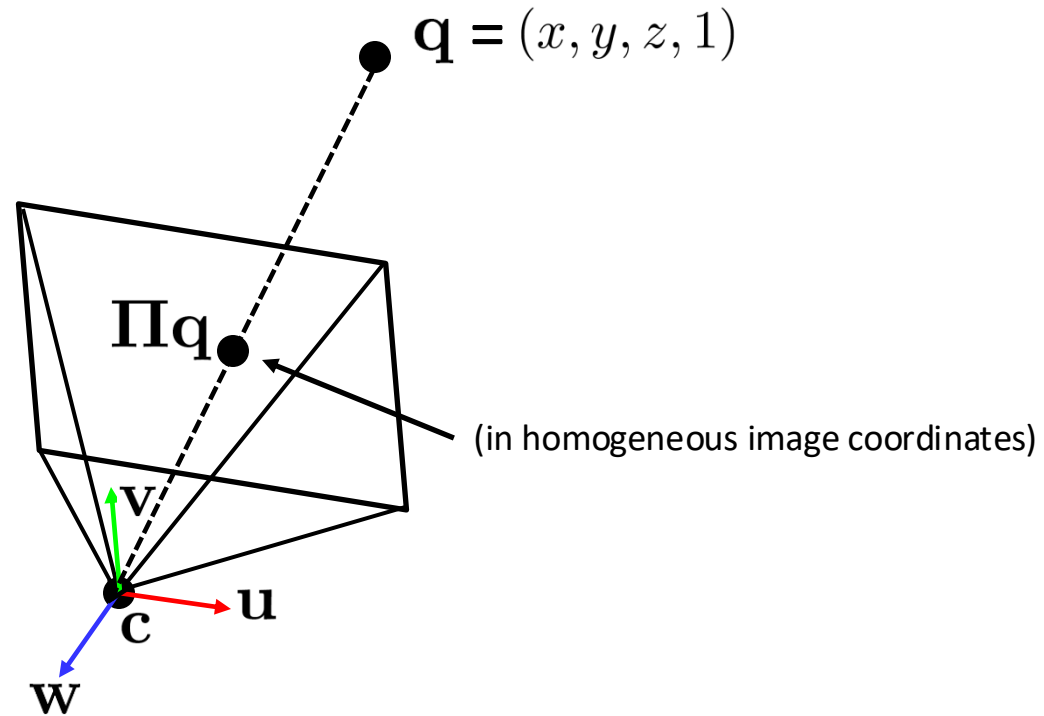
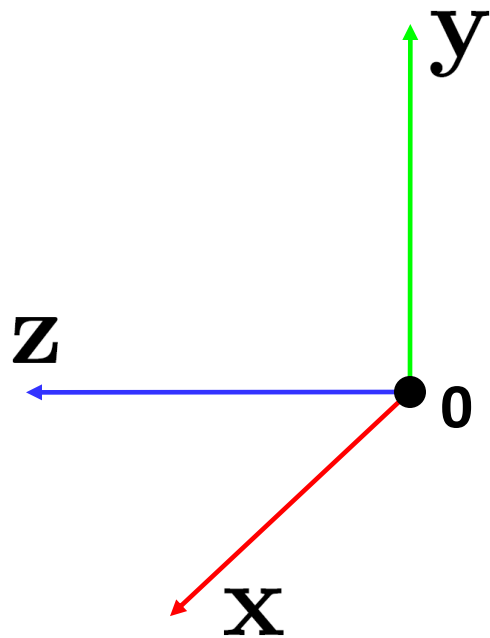
$$\left[\mathbf{R} \mid \underbrace{-\mathbf{R}\mathbf{c}} \right]$$

($\mathbf{t}$ in book's notation)



$$\mathbf{\Pi} = \mathbf{K} \left[\mathbf{R} \mid -\mathbf{R}\mathbf{c} \right]$$

Projection matrix



How to calibrate the camera?
(also called “camera resectioning”)

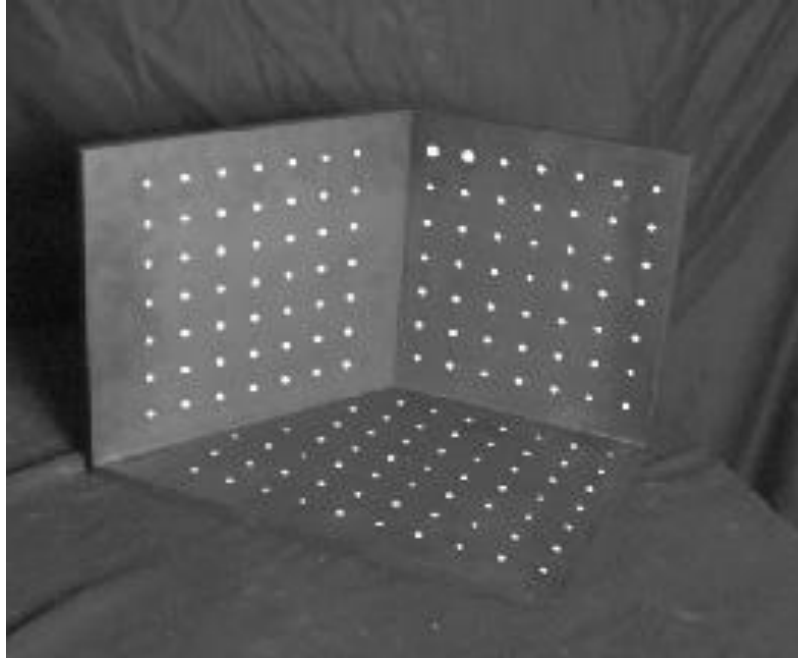
$$\mathbf{x} = \mathbf{K} \begin{bmatrix} \mathbf{R} & \mathbf{t} \end{bmatrix} \mathbf{X}$$

$$\begin{bmatrix} su \\ sv \\ s \end{bmatrix} = \begin{bmatrix} * & * & * & * \\ * & * & * & * \\ * & * & * & * \end{bmatrix} \begin{bmatrix} X \\ Y \\ Z \\ 1 \end{bmatrix}$$

Calibrating the Camera

Use an scene with known geometry

- Correspond image points to 3d points
- Get least squares solution (or non-linear solution)



Known 2d
image coords



$$\begin{bmatrix} su \\ sv \\ s \end{bmatrix} = \begin{bmatrix} m_{11} & m_{12} & m_{13} & m_{14} \\ m_{21} & m_{22} & m_{23} & m_{24} \\ m_{31} & m_{32} & m_{33} & m_{34} \end{bmatrix} \begin{bmatrix} X \\ Y \\ Z \\ 1 \end{bmatrix}$$

Known 3d
locations

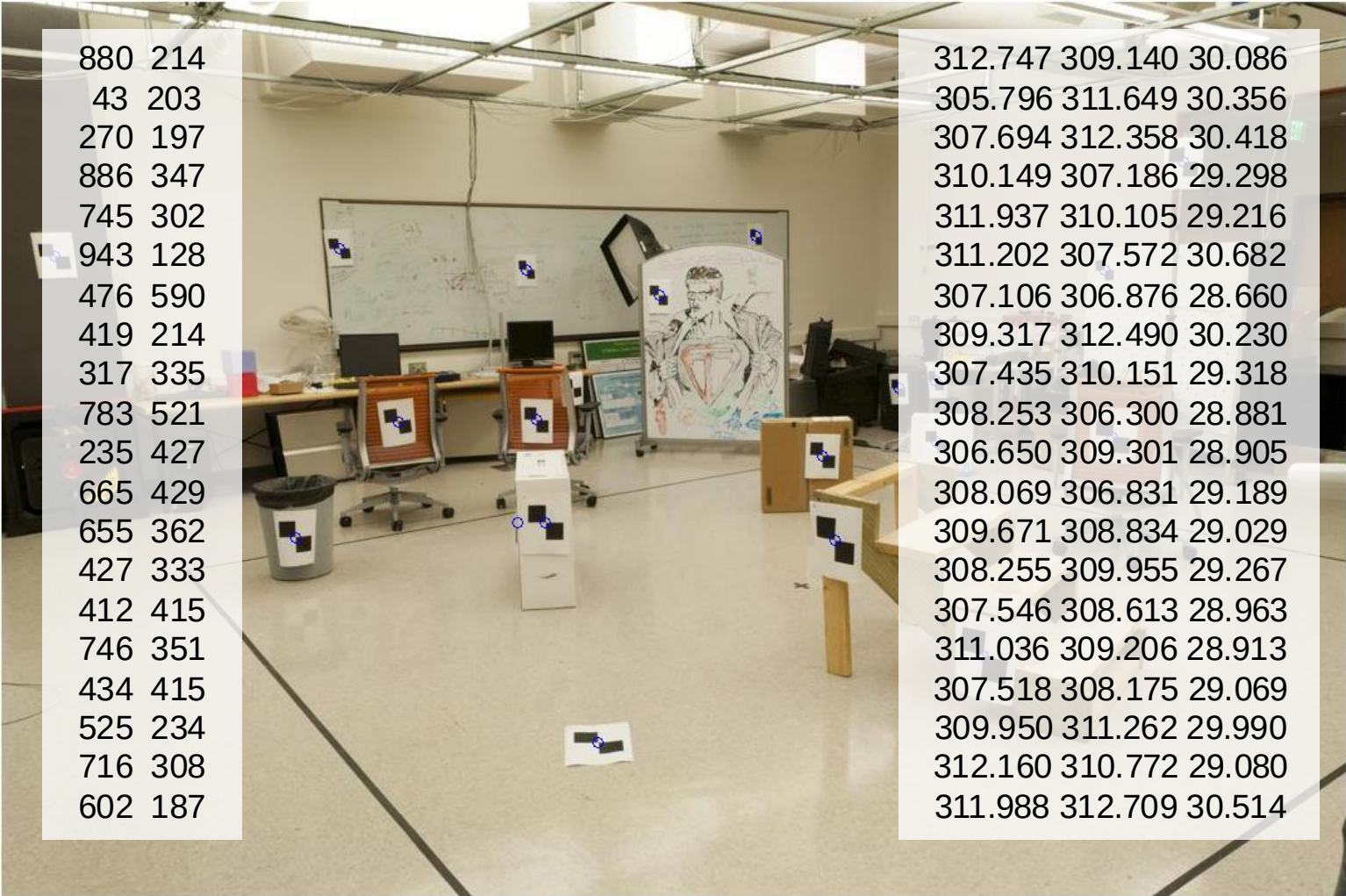


Unknown Camera Parameters

How do we calibrate a camera?

Known 2d
image coords

Known 3d
locations



880	214
43	203
270	197
886	347
745	302
943	128
476	590
419	214
317	335
783	521
235	427
665	429
655	362
427	333
412	415
746	351
434	415
525	234
716	308
602	187

312.747	309.140	30.086
305.796	311.649	30.356
307.694	312.358	30.418
310.149	307.186	29.298
311.937	310.105	29.216
311.202	307.572	30.682
307.106	306.876	28.660
309.317	312.490	30.230
307.435	310.151	29.318
308.253	306.300	28.881
306.650	309.301	28.905
308.069	306.831	29.189
309.671	308.834	29.029
308.255	309.955	29.267
307.546	308.613	28.963
311.036	309.206	28.913
307.518	308.175	29.069
309.950	311.262	29.990
312.160	310.772	29.080
311.988	312.709	30.514

Unknown Camera Parameters



Known 2d
image coords

$$\begin{bmatrix} su \\ sv \\ s \end{bmatrix} = \begin{bmatrix} m_{11} & m_{12} & m_{13} & m_{14} \\ m_{21} & m_{22} & m_{23} & m_{24} \\ m_{31} & m_{32} & m_{33} & m_{34} \end{bmatrix} \begin{bmatrix} X \\ Y \\ Z \\ 1 \end{bmatrix}$$

Known 3d
locations

$$su = m_{11}X + m_{12}Y + m_{13}Z + m_{14}$$

$$sv = m_{21}X + m_{22}Y + m_{23}Z + m_{24}$$

$$s = m_{31}X + m_{32}Y + m_{33}Z + m_{34}$$

$$u = \frac{m_{11}X + m_{12}Y + m_{13}Z + m_{14}}{m_{31}X + m_{32}Y + m_{33}Z + m_{34}}$$

$$v = \frac{m_{21}X + m_{22}Y + m_{23}Z + m_{24}}{m_{31}X + m_{32}Y + m_{33}Z + m_{34}}$$

Unknown Camera Parameters



Known 2d
image coords

$$\begin{bmatrix} su \\ sv \\ s \end{bmatrix} = \begin{bmatrix} m_{11} & m_{12} & m_{13} & m_{14} \\ m_{21} & m_{22} & m_{23} & m_{24} \\ m_{31} & m_{32} & m_{33} & m_{34} \end{bmatrix} \begin{bmatrix} X \\ Y \\ Z \\ 1 \end{bmatrix}$$

Known 3d
locations

$$u = \frac{m_{11}X + m_{12}Y + m_{13}Z + m_{14}}{m_{31}X + m_{32}Y + m_{33}Z + m_{34}}$$

$$v = \frac{m_{21}X + m_{22}Y + m_{23}Z + m_{24}}{m_{31}X + m_{32}Y + m_{33}Z + m_{34}}$$

$$(m_{31}X + m_{32}Y + m_{33}Z + m_{34})u = m_{11}X + m_{12}Y + m_{13}Z + m_{14}$$

$$(m_{31}X + m_{32}Y + m_{33}Z + m_{34})v = m_{21}X + m_{22}Y + m_{23}Z + m_{24}$$

$$m_{31}uX + m_{32}uY + m_{33}uZ + m_{34}u = m_{11}X + m_{12}Y + m_{13}Z + m_{14}$$

$$m_{31}vX + m_{32}vY + m_{33}vZ + m_{34}v = m_{21}X + m_{22}Y + m_{23}Z + m_{24}$$

Unknown Camera Parameters



Known 2d image coords

$$\begin{bmatrix} su \\ sv \\ s \end{bmatrix} = \begin{bmatrix} m_{11} & m_{12} & m_{13} & m_{14} \\ m_{21} & m_{22} & m_{23} & m_{24} \\ m_{31} & m_{32} & m_{33} & m_{34} \end{bmatrix} \begin{bmatrix} X \\ Y \\ Z \\ 1 \end{bmatrix}$$

Known 3d locations

$$m_{31}uX + m_{32}uY + m_{33}uZ + m_{34}u = m_{11}X + m_{12}Y + m_{13}Z + m_{14}$$

$$m_{31}vX + m_{32}vY + m_{33}vZ + m_{34}v = m_{21}X + m_{22}Y + m_{23}Z + m_{24}$$

$$0 = m_{11}X + m_{12}Y + m_{13}Z + m_{14} - m_{31}uX - m_{32}uY - m_{33}uZ - m_{34}u$$

$$0 = m_{21}X + m_{22}Y + m_{23}Z + m_{24} - m_{31}vX - m_{32}vY - m_{33}vZ - m_{34}v$$

Unknown Camera Parameters



Known 2d
image coords

$$\begin{bmatrix} su \\ sv \\ s \end{bmatrix} = \begin{bmatrix} m_{11} & m_{12} & m_{13} & m_{14} \\ m_{21} & m_{22} & m_{23} & m_{24} \\ m_{31} & m_{32} & m_{33} & m_{34} \end{bmatrix} \begin{bmatrix} X \\ Y \\ Z \\ 1 \end{bmatrix}$$

Known 3d
locations

$$0 = m_{11}X + m_{12}Y + m_{13}Z + m_{14} - m_{31}uX - m_{32}uY - m_{33}uZ - m_{34}u$$

$$0 = m_{21}X + m_{22}Y + m_{23}Z + m_{24} - m_{31}vX - m_{32}vY - m_{33}vZ - m_{34}v$$

For each point i you have two equations

$$\begin{bmatrix} X_i & Y_i & Z_i & 1 & 0 & 0 & 0 & 0 & -u_iX_i & -u_iY_i & -u_iZ_i & -u_i \\ 0 & 0 & 0 & 0 & X_i & Y_i & Z_i & 1 & -v_iX_i & -v_iY_i & -v_iZ_i & -v_i \end{bmatrix} \cdot \begin{bmatrix} m_{11} \\ m_{12} \\ m_{13} \\ m_{14} \\ m_{21} \\ m_{22} \\ m_{23} \\ m_{24} \\ m_{31} \\ m_{32} \\ m_{33} \\ m_{34} \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

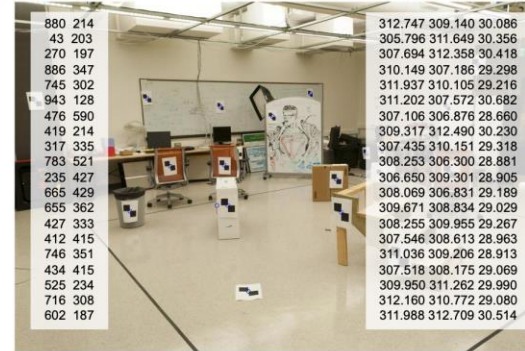
Stack these for **N points** → you get a matrix **A**, size $2N \times 12$:

$$A \cdot m = 0$$

Camera Calibration

Known 2d
image coords

Known 3d
locations



$$\begin{bmatrix}
 X_1 & Y_1 & Z_1 & 1 & 0 & 0 & 0 & 0 & -u_1 X_1 & -u_1 Y_1 & -u_1 Z_1 & -u_1 \\
 0 & 0 & 0 & 0 & X_1 & Y_1 & Z_1 & 1 & -v_1 X_1 & -v_1 Y_1 & -v_1 Z_1 & -v_1 \\
 & & & & \vdots & & & & & & & \\
 X_n & Y_n & Z_n & 1 & 0 & 0 & 0 & 0 & -u_n X_n & -u_n Y_n & -u_n Z_n & -u_n \\
 0 & 0 & 0 & 0 & X_n & Y_n & Z_n & 1 & -v_n X_n & -v_n Y_n & -v_n Z_n & -v_n
 \end{bmatrix}
 \begin{bmatrix}
 m_{11} \\
 m_{12} \\
 m_{13} \\
 m_{14} \\
 m_{21} \\
 m_{22} \\
 m_{23} \\
 m_{24} \\
 m_{31} \\
 m_{32} \\
 m_{33} \\
 m_{34}
 \end{bmatrix}
 =
 \begin{bmatrix}
 0 \\
 0 \\
 \vdots \\
 0 \\
 0
 \end{bmatrix}$$

m is a **12-dimensional vector** representing all entries in M

$$A \cdot m = 0$$

Solve for m Using SVD (Singular Value Decomposition)

You need the **non-trivial solution** to

$$Am = 0$$

- Trivial solution $m = 0$ isn't useful.
- So you **minimize** $\|Am\|$, subject to $\|m\| = 1$.

This is a **constrained optimization problem**, and its solution is the **right singular vector** of A corresponding to the **smallest singular value**.

But How?

SVD Factorization

$$A = U D V^T$$

where:

- U and V are orthogonal matrices (their columns/rows are orthonormal vectors).
- D is diagonal with non-negative entries ($\sigma_1 \geq \sigma_2 \geq \dots \geq \sigma_n$), the singular values of A .

Objective: We want to minimize $\|A m\|$ subject to $\|m\| = 1$

Orthogonality Simplifies Norms: Because U is orthonormal, $\|U(\cdot)\| = \|(\cdot)\|$. Hence, $\|U D V^T m\| = \|D V^T m\|$.

Let $y = V^T m$. Since V is orthonormal, $\|m\| = 1$ implies $\|y\| = 1$ as well.

Note: For any **orthonormal** (unitary) matrix Q , we have $Q^T Q = I$.

Hence, $\|Q \mathbf{x}\|^2 = (Q \mathbf{x})^T (Q \mathbf{x}) = \mathbf{x}^T \mathbf{x} = \|\mathbf{x}\|^2$,

so $\|Q \mathbf{x}\| = \|\mathbf{x}\|$.

SVD Factorization

Objective: We want to minimize $\|A m\|$ subject to $\|m\| = 1$

$$A = U D V^T$$

$$\|U D V^T m\| = \|D V^T m\| = \|D y\|$$

Modified Objective:

Minimize $\|D y\|$ subject to $\|y\| = 1$

$$D = \begin{bmatrix} \sigma_1 & 0 & 0 & \dots & 0 \\ 0 & \sigma_2 & 0 & \dots & 0 \\ 0 & 0 & \sigma_3 & \dots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \dots & \sigma_n \end{bmatrix} \quad y = \begin{bmatrix} 0 \\ 0 \\ \vdots \\ 1 \end{bmatrix}$$

$$\sigma_1 \geq \sigma_2 \geq \dots \geq \sigma_n \geq 0$$

Back-Substitute

Since $y = V^T m \implies m = V y$,

the optimal m is the last column of V , corresponding to σ_n .

$$m = V y = \begin{bmatrix} | & | & \dots & | \\ \mathbf{v}_1 & \mathbf{v}_2 & \dots & \mathbf{v}_n \\ | & | & \dots & | \end{bmatrix} \cdot \begin{bmatrix} 0 \\ 0 \\ \vdots \\ 1 \end{bmatrix} = \mathbf{v}_n$$

This last column corresponds to the **smallest singular value** direction (assuming σ_n is the smallest).

Algorithm: Find Camera Projection Matrix M

1. Collect 3D–2D Correspondences

- For each 3D point $\mathbf{X}_i$ and its 2D projection $\mathbf{x}_i$, form two homogeneous equations.

2. Assemble Matrix A

- Stack all equations into a $2N \times 12$ matrix A .
- Vectorize the unknowns $m = (m_{11}, \dots, m_{34})^T$.

3. Solve $A m = 0$ via SVD

- Perform $\text{SVD}(A) = U D V^T$.
- The **last column** of V (or equivalently the last row of V^T) gives the **best** m .

4. Reshape m into M

- Convert the 12-vector back into a 3×4 matrix M .

Python Code

```
U, S, Vt = np.linalg.svd(A)
m = Vt[-1, :]
M = m.reshape(3, 4)
```

METHOD 2

Scale Ambiguity in $\mathbf{M}$

Known 2d
image coords



$$\begin{bmatrix} su \\ sv \\ s \end{bmatrix}$$

$\mathbf{M}$

$$= \begin{bmatrix} m_{11} & m_{12} & m_{13} & m_{14} \\ m_{21} & m_{22} & m_{23} & m_{24} \\ m_{31} & m_{32} & m_{33} & m_{34} \end{bmatrix}$$

Known 3d
locations



$$\begin{bmatrix} X \\ Y \\ Z \\ 1 \end{bmatrix}$$

Any scalar multiple $\lambda \mathbf{M}$ projects the same points.
→ If $\mathbf{M}$ works, so does $\lambda \mathbf{M}$.

- This **scale ambiguity** means you can **arbitrarily set** one element of $\mathbf{M}$ (commonly m_{34}) to a fixed value
- This value is usually **1**
- This **fixes the scale** and make the system solvable.

Method 2

- We write the camera matrix $M = \begin{bmatrix} m_{11} & m_{12} & m_{13} & m_{14} \\ m_{21} & m_{22} & m_{23} & m_{24} \\ m_{31} & m_{32} & m_{33} & 1 \end{bmatrix}$

For a 3D point (X_i, Y_i, Z_i) mapping to image (u_i, v_i) , the pinhole model becomes:

$$\begin{bmatrix} s u_i \\ s v_i \\ s \end{bmatrix} = \begin{bmatrix} m_{11} & m_{12} & m_{13} & m_{14} \\ m_{21} & m_{22} & m_{23} & m_{24} \\ m_{31} & m_{32} & m_{33} & 1 \end{bmatrix} \begin{bmatrix} X_i \\ Y_i \\ Z_i \\ 1 \end{bmatrix}.$$

Dividing through by s (or rearranging terms) gives two equations per point:

$$u_i = \frac{m_{11}X_i + m_{12}Y_i + m_{13}Z_i + m_{14}}{m_{31}X_i + m_{32}Y_i + m_{33}Z_i + 1},$$
$$v_i = \frac{m_{21}X_i + m_{22}Y_i + m_{23}Z_i + m_{24}}{m_{31}X_i + m_{32}Y_i + m_{33}Z_i + 1}.$$

Method 2

- Nonhomogeneous linear system.
- Solve for m's entries using linear least squares

$$m_{11}X + m_{12}Y + m_{13}Z + m_{14} - m_{31}uX - m_{32}uY - m_{33}uZ = u,$$

$$m_{21}X + m_{22}Y + m_{23}Z + m_{24} - m_{31}vX - m_{32}vY - m_{33}vZ = v.$$

$$\begin{bmatrix} X_1 & Y_1 & Z_1 & 1 & 0 & 0 & 0 & 0 & -u_1X_1 & -u_1Y_1 & -u_1Z_1 \\ 0 & 0 & 0 & 0 & X_1 & Y_1 & Z_1 & 1 & -v_1X_1 & -v_1Y_1 & -v_1Z_1 \\ & & & & & & \vdots & & & & \\ X_n & Y_n & Z_n & 1 & 0 & 0 & 0 & 0 & -u_nX_n & -u_nY_n & -u_nZ_n \\ 0 & 0 & 0 & 0 & X_n & Y_n & Z_n & 1 & -v_nX_n & -v_nY_n & -v_nZ_n \end{bmatrix} \begin{bmatrix} m_{11} \\ m_{12} \\ m_{13} \\ m_{14} \\ m_{21} \\ m_{22} \\ m_{23} \\ m_{24} \\ m_{31} \\ m_{32} \\ m_{33} \end{bmatrix} = \begin{bmatrix} u_1 \\ v_1 \\ \vdots \\ u_n \\ v_n \end{bmatrix}$$

Already covered: Unknown Camera Parameters {Method 1}



Known 2d
image coords

$$\begin{bmatrix} su \\ sv \\ s \end{bmatrix} = \begin{bmatrix} m_{11} & m_{12} & m_{13} & m_{14} \\ m_{21} & m_{22} & m_{23} & m_{24} \\ m_{31} & m_{32} & m_{33} & m_{34} \end{bmatrix} \begin{bmatrix} X \\ Y \\ Z \\ 1 \end{bmatrix}$$

Known 3d
locations

$$0 = m_{11}X + m_{12}Y + m_{13}Z + m_{14} - m_{31}uX - m_{32}uY - m_{33}uZ - m_{34}u$$

$$0 = m_{21}X + m_{22}Y + m_{23}Z + m_{24} - m_{31}vX - m_{32}vY - m_{33}vZ - m_{34}v$$

$$\begin{bmatrix} X_1 & Y_1 & Z_1 & 1 & 0 & 0 & 0 & 0 & -u_1X_1 & -u_1Y_1 & -u_1Z_1 & -u_1 \\ 0 & 0 & 0 & 0 & X_1 & Y_1 & Z_1 & 1 & -v_1X_1 & -v_1Y_1 & -v_1Z_1 & -v_1 \\ & & & & & & \vdots & & & & & \\ X_n & Y_n & Z_n & 1 & 0 & 0 & 0 & 0 & -u_nX_n & -u_nY_n & -u_nZ_n & -u_n \\ 0 & 0 & 0 & 0 & X_n & Y_n & Z_n & 1 & -v_nX_n & -v_nY_n & -v_nZ_n & -v_n \end{bmatrix} \begin{bmatrix} m_{11} \\ m_{12} \\ m_{13} \\ m_{14} \\ m_{21} \\ m_{22} \\ m_{23} \\ m_{24} \\ m_{31} \\ m_{32} \\ m_{33} \\ m_{34} \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ \vdots \\ 0 \\ 0 \end{bmatrix}$$

*** Unknown Camera Parameters {Method 2} ***



Known 2d
image coords

$$\begin{bmatrix} s u_i \\ s v_i \\ s \end{bmatrix} = \begin{bmatrix} m_{11} & m_{12} & m_{13} & m_{14} \\ m_{21} & m_{22} & m_{23} & m_{24} \\ m_{31} & m_{32} & m_{33} & 1 \end{bmatrix} \begin{bmatrix} X_i \\ Y_i \\ Z_i \\ 1 \end{bmatrix}$$

Known 3d
locations

Am=b form

$$\begin{bmatrix} X_1 & Y_1 & Z_1 & 1 & 0 & 0 & 0 & 0 & -u_1 X_1 & -u_1 Y_1 & -u_1 Z_1 \\ 0 & 0 & 0 & 0 & X_1 & Y_1 & Z_1 & 1 & -v_1 X_1 & -v_1 Y_1 & -v_1 Z_1 \\ & & & & \vdots & & & & & & \\ X_n & Y_n & Z_n & 1 & 0 & 0 & 0 & 0 & -u_n X_n & -u_n Y_n & -u_n Z_n \\ 0 & 0 & 0 & 0 & X_n & Y_n & Z_n & 1 & -v_n X_n & -v_n Y_n & -v_n Z_n \end{bmatrix} \begin{bmatrix} m_{11} \\ m_{12} \\ m_{13} \\ m_{14} \\ m_{21} \\ m_{22} \\ m_{23} \\ m_{24} \\ m_{31} \\ m_{32} \\ m_{33} \end{bmatrix} = \begin{bmatrix} u_1 \\ v_1 \\ \vdots \\ u_n \\ v_n \end{bmatrix}$$

In python,

```
m = np.linalg.pinv(A) @ b
m = np.concatenate([m, [1]])
M = m.reshape(3, 4)
```

Calibration with Linear Method

- Advantages

- Easy to formulate and solve. You simply assemble a matrix A from your 3D–2D correspondences and solve $Am = 0$ or $Am = b$ using linear algebra.
- Provides initialization for non-linear methods

- Disadvantages

- Doesn't directly give you camera parameters, f, c_x, c_y
- Doesn't model radial distortion. Assume an ideal “pinhole” projection
- Can't impose constraints, such as known focal length f

- Non-linear methods are preferred

- Define a **reprojection error**: the difference between the **measured** 2D points and the **projected** 2D points (from 3D).
- Use iterative optimization (Newton's method, Levenberg–Marquardt, etc.) to **minimize** this error.
- Though, Nonlinear methods need a good initial guess to avoid local minima. That's why the linear solution is often used first.

We will not cover Non-linear methods

Recovering the camera center

$$\mathbf{x} = \mathbf{K}[\mathbf{R} \quad \mathbf{t}] \mathbf{X}$$

$$w \begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = \begin{bmatrix} \alpha & s & u_0 \\ 0 & \beta & v_0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} r_{11} & r_{12} & r_{13} \\ r_{21} & r_{22} & r_{23} \\ r_{31} & r_{32} & r_{33} \end{bmatrix} \begin{bmatrix} t_x \\ t_y \\ t_z \end{bmatrix} \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$

This is not the camera center $-C$. It is $-\mathbf{RC}$ (because a point will be rotated before t_x , t_y , and t_z are added)

This is $\mathbf{K} * \mathbf{t}$
So $\mathbf{K}^{-1} m_4$ is $\mathbf{t}$

$$\begin{bmatrix} su \\ sv \\ s \end{bmatrix} = \underbrace{\begin{bmatrix} * & * & * \\ * & * & * \\ * & * & * \end{bmatrix}}_{\mathbf{Q}} \begin{bmatrix} X \\ Y \\ Z \\ 1 \end{bmatrix}$$

So we need $-\mathbf{R}^{-1} \mathbf{K}^{-1} m_4$ to get C

$\mathbf{Q}$ is $\mathbf{K} * \mathbf{R}$. So we just need $-\mathbf{Q}^{-1} m_4$